CHANAKYA UNIVERSITY

A PROJECT DEVELOPMENT DOCUMENTATION

PRJ-18

REAL-TIME UNIVERSITY BUS TRACKING SYSTEM

Project Development Documentation — Week 5 Revision

Living document updated to reflect verified progress through Week 5. Completed, in-progress, and planned items are kept distinct; pending features are not presented as complete.

Master of Computer Applications (MCA)

Submitted by

Team Size: 2 Members
Team Member 1: Birendra Prasad — 25PG00018 (MCA-CBS)
Team Member 2: Meghana R — 25PG00174(MCA-DS)

Under the Guidance of

Sarath Babu Sribhasyam Sir

Backend: Python + FastAPI | Frontend: HTML + CSS + JavaScript
Database: PostgreSQL | Real-Time Streaming: Apache Kafka (KRaft)
Analytics: Apache Superset (Planned) | Map: Leaflet + OpenStreetMap
Version Control: Git + GitHub

Repository: https://github.com/BirendraPrasad/University-Bus-Tracking

Table of Contents

Document Purpose

1. Project Overview
2. Project Objectives
3. User Roles
4. What We Have Completed
5. Backend Development
6. Database
7. System Architecture
8. Current Data Flow
9. Login Flow
10. Admin Flow
11. Student Flow
12. Driver Flow
13. Complete Real-Time System Flow (Target / Planned)
14. API Documentation
15. Testing Completed (Through Week 4)
16. Current Status
17. Development Roadmap
18. What We Should Do Next
19. Week 5 — Kafka Integration, GPS Data Persistence, and Student API Improvements

Appendix A — Week 5 Changelog

Appendix B — Screenshots and Evidence to Capture Before Submission

Appendix C — Security and Data-Handling Reminders

Appendix D — Project Implementation Screenshot Evidence

List of Figures

Figure 1: Entity-Relationship diagram for university_bus_tracking, updated Week 5 — implemented/verified vs. planned tables clearly marked.

Figure 2: Overall system architecture — the implemented request/response path.

Figure 3: Planned real-time location pipeline — original target architecture.

Figure 4: Planned analytics pipeline via Apache Superset.

Figure 5: Data flow diagram for the currently implemented system.

Figure 6: Login flow — authentication and authorization.

Figure 7: Admin flow: authentication, authorization, and the management APIs it unlocks.

Figure 8: Student flow, with each section marked by its real implementation status.

Figure 9: Planned driver flow, from login through to GPS reaching the live map.

Figure 10: GPS data flow — original target design.

Figure 11: Kafka streaming flow — original target design (see Figure 15 in Chapter 19 for the Week 5 verified version).

Figure 12: ETA calculation flow — target design, not yet implemented.

Figure 13: Complete target system flow — end-to-end target architecture.

Figure 14: Twelve-phase development roadmap, updated with Week 5 progress.

Figure 15: Week 5 verified Kafka-to-database pipeline. The driver-phone, GPS-capture, FastAPI-producer, live-map, and ETA stages remain planned; the Kafka broker, topic, consumer, and database write are verified.

List of Tables

Table 1: Project objectives and current status (updated Week 5).

Table 2: Role responsibilities and current implementation status.

Table 3: Repository and project-structure work completed.

Table 4: Environment, backend, and database configuration work completed.

Table 5: Data models and management API work completed.

Table 6: Authentication, authorization, and API documentation work completed.

Table 7: Frontend work completed.

Table 8: Purpose of each backend folder.

Table 9: Database tables, purpose, and implementation status (updated Week 5).

Table 10: Student dashboard sections and their verified status as of Week 5.

Table 11: Implemented and verified API endpoints, with purpose, authentication requirement, and status (updated Week 5).

Table 12: Manual tests performed through Week 4, executed against the implemented backend and frontend.

Table 13: Feature-by-feature project status as of Week 5.

Table 14: Development roadmap phases and current status (updated Week 5).

Table 15: Week 5 Kafka configuration and rationale.

Table 16: Student endpoints verified in Week 5 and their role in the overall system.

Table 17: Week 5 testing evidence. Expected result for each row is the corresponding implemented behaviour described in the Test Objective column.

Table 18: Summary of sections added or updated for the Week 5 revision.

Table 19: Screenshot and evidence checklist for Week 5 submission.

Document Purpose

This document is a concise, accurate, and continuously updated development record for PRJ-18, the Real-Time University Bus Tracking System. It is not a final academic thesis. Its purpose is to give any evaluator, mentor, or new team member three clear answers: what has actually been developed and verified so far; how the complete system is designed and intended to work end-to-end; and what remains to be built, in what order.

Throughout this document, every component, API, table, and diagram element is explicitly tagged with one of five statuses. COMPLETED and VERIFIED are kept distinct on purpose: a component can be built (COMPLETED) before its end-to-end behaviour has actually been exercised and observed (VERIFIED) — the two claims are not interchangeable.

Status Tag	Meaning
COMPLETED	Implementation completed — the component is built and connected.
VERIFIED	Specifically tested and observed to behave as described, with evidence in this document.
IN PROGRESS	Partially implemented — some parts exist, others do not yet.
PLANNED / PENDING	Not implemented; must not be claimed as complete.
NOT VERIFIED	Implementation or behaviour exists in some form but has not been confirmed this revision; must not be read as a pass.

> *Revision note (Week 5): earlier revisions of this document described Kafka and GPS persistence as entirely planned. Development notes now show the Kafka broker and topic were verified, the GPS consumer received events, and location rows were observed in PostgreSQL. The student live map, ETA, timetable, and student notifications remain pending or not verified end-to-end — see Chapter 19 for full detail.*

1. Project Overview

PRJ-18, the Real-Time University Bus Tracking System, is a web-based platform being built to help students, faculty, drivers, and administrators manage and monitor university bus transport from a single system. In its target form, the system will let a driver's phone report the live location of a bus, and let students and faculty see that bus moving on a map along with an estimated time of arrival at their stop.

The problem it solves is a familiar one on university campuses: students and faculty currently have no reliable way to know where a bus is, whether it is delayed, or when it will actually arrive at their stop. This leads to wasted waiting time, missed buses, and no central visibility for transport administrators into how the fleet is being used. A real-time tracking system removes this uncertainty by making bus location, route, and schedule information available digitally and, eventually, live.

Real-time tracking is especially useful in a university context because bus schedules are shared across a large number of routes and stops, ridership is high during specific class-change windows, and the administration needs data (not guesswork) to plan routes, assign drivers, and respond to delays.

When complete, the final system will provide: role-based dashboards for students, faculty, drivers, and administrators; a searchable directory of buses, routes, and stops; a live map showing each bus's current position; an estimated time of arrival (ETA) for each stop; and an analytics dashboard for administrators built on the data the system collects. As of this revision (Week 5), the management, authentication, and dashboard-UI portions of this vision are implemented, and the Kafka-to-database half of the real-time pipeline has been verified; the GPS-capture, live-map, and ETA portions are still pending — this distinction is maintained throughout the document.

2. Project Objectives

The project is being developed against the following objectives. Each objective is tagged with its current status so the reader does not need to infer progress from prose.

Objective	Description	Status
Manage university buses	Create, list, and update bus records (registration number, capacity, assigned driver).	**COMPLETED**

Objective	Description	Status
Manage routes	Create and list routes that buses operate on.	COMPLETED
Manage stops	Create stops and attach them to a route in sequence.	COMPLETED
Manage drivers	Register drivers and assign them to buses.	COMPLETED
Authenticate users	Secure login using hashed passwords and JWT tokens.	COMPLETED
Provide role-based access	Restrict admin-only and driver-only endpoints by role.	COMPLETED
Let students/faculty view bus information	Dashboards to browse buses, routes, and stops.	IN PROGRESS
Persist GPS events via Kafka	Move location updates from an event source to the database via a Kafka pipeline.	IN PROGRESS
Provide live bus tracking	Show a bus's real-time position on a map.	PLANNED
Calculate ETA	Estimate arrival time at each stop from live position and speed.	PLANNED
Capture GPS from the driver's device	Browser Geolocation API feeding a FastAPI producer endpoint.	PLANNED
Provide transport analytics using Superset	Route/trip/ETA metrics for administrators.	PLANNED

Table 1: Project objectives and current status (updated Week 5).

> *As of Week 5, the Kafka broker, the bus-location topic, the GPS consumer, and PostgreSQL persistence have been verified working together. Driver-side GPS capture, the FastAPI producer endpoint, the live student map, ETA calculation, and Superset analytics remain PLANNED. See Chapter 19 for full detail.*

3. User Roles

The system is designed around four conceptual user roles: Student, Driver, Faculty, and Admin. This four-role model is a design decision made by our team for PRJ-18 — it is not dictated by any external specification — chosen because it maps cleanly onto how a university transport system is actually used: two groups of riders (students and faculty) with near-identical needs, one operational role (driver) that generates data, and one administrative role that manages the whole system.

Role	Responsibilities	Backend Status	Frontend Status
Student	Login, Dashboard, My Routes, Live Tracking, Bus Details, Timetable, Notifications, Profile	Core profile/route/stop/bus-details APIs verified; timetable and notifications pending	Dashboard UI built
Driver	Login, Assigned Bus, Assigned Route, Start Trip, GPS Sharing, Trip History	Dashboard/trip screens shown; GPS producer pipeline pending	Dashboard screens demonstrated
Faculty	Login, Routes, Live Tracking, Bus Details, Timetable, Notifications, Profile	Shares Student-level APIs	Not yet built
Admin	User management, Driver management, Bus management, Route management, Stop management, Trip management, Live monitoring, Reports, Analytics	Management APIs implemented; monitoring/reports planned	Not yet built (Swagger used for now)

Table 2: Role responsibilities and current implementation status.

The university's fleet consists of nine buses in total: six route buses and three shuttles. Development and testing to date have used a single bus/route/trip combination to validate the pipeline end-to-end; this does not represent the complete, officially configured fleet. Full fleet configuration depends on the Admin CRUD dashboard (Chapter 17, Phase 10) and on official route/timing data being entered once that tool exists.

4. What We Have Completed

This chapter lists only the work that has actually been built and verified to run. Items are grouped into five categories for readability. Each table states what was done, the technology used, and the current, observable result. Week 5 additions (Kafka and the student API fix) are documented separately in Chapter 19 to keep this chapter focused on Weeks 1–4 foundational work.

4.1 Repository, Project Structure & Version Control

A clean repository structure was established first so that backend, frontend, database, and future Kafka/Superset work stay isolated and easy to navigate as the team grows the codebase.

Item	What Was Done	Technology	Current Result
GitHub repository	Central repository created and shared between both team members.	Git + GitHub	Repo live at github.com/BirendraPrasad/University-Bus-Tracking
Project folder structure	Top-level folders separated by concern.	Filesystem convention	backend/, frontend/, database/, kafka/, superset/ exist

Item	What Was Done	Technology	Current Result
Backend folder	Dedicated FastAPI application folder.	FastAPI project layout	Contains app/ with models, schemas, routers, services
Frontend folder	Static HTML/CSS/JS app folder.	Plain HTML/CSS/JS	Contains landing, login, and dashboard pages
Database folder	Holds schema/SQL assets for PostgreSQL.	SQL	Schema scripts kept alongside code
Kafka folder	Holds the GPS consumer script and related Kafka assets.	Apache Kafka	Now contains a working gps_consumer.py (Chapter 19)
Superset folder (placeholder)	Reserved location for future dashboard config/export.	Apache Superset (planned)	Folder exists; empty of working code
Git commit history	Regular commits pushed as features were completed.	Git + GitHub	Incremental commit history visible on GitHub

Table 3: Repository and project-structure work completed.

4.2 Environment, Backend & Database Configuration

Before any feature could be built, the local development environment, the FastAPI server, and the PostgreSQL database had to be installed, configured, and connected to each other.

Item	What Was Done	Technology	Current Result
Python virtual environment	Isolated environment created for backend dependencies.	venv	Reproducible local backend setup
FastAPI installed/configured	Core web framework installed and app entry point created.	FastAPI	App boots and serves requests
Uvicorn configured	ASGI server configured to run the FastAPI app.	Uvicorn	Server runs locally with auto-reload
PostgreSQL installed/configured	Database server installed locally.	PostgreSQL	Server running and reachable
PostgreSQL database created	Dedicated database created for the project.	PostgreSQL	university_bus_tracking database exists
PostgreSQL user/role configured	Application-specific DB user with scoped privileges.	PostgreSQL roles	Backend connects using a dedicated role, not superuser
Database schema created	Core tables created (see Chapter 6).	SQL / SQLAlchemy models	Schema present in the database
FastAPI connected with PostgreSQL	ORM/DB session wired into the FastAPI app.	SQLAlchemy	Endpoints read/write real DB rows
.env configuration	Secrets and connection strings externalised from code.	python-dotenv	DB URL, JWT secret loaded from .env
.gitignore configured	Prevents venv, .env, and cache files from being committed.	Git	Repository stays clean of local artefacts

Table 4: Environment, backend, and database configuration work completed.

4.3 Data Models & Management APIs

With the database connected, core entities were modelled and exposed through REST endpoints so that buses, routes, stops, and people involved in transport can be created and queried.

Item	What Was Done	Technology	Current Result
User model/schema	Base user table + Pydantic schemas for request/response validation.	SQLAlchemy + Pydantic	Shared identity for all four roles
Student management	Create/list student records linked to a user.	FastAPI router + service	Students can be registered via API
Driver management	Create/list driver records linked to a user.	FastAPI router + service	Drivers can be registered via API
Faculty management	Create/list faculty records linked to a user.	FastAPI router + service	Faculty can be registered via API
Bus management	Create/list buses.	FastAPI router + service	Buses can be created and fetched
Route management	Create/list routes.	FastAPI router + service	Routes can be created and fetched
Stop management	Create stops under a route; list stops per route.	FastAPI router + service	Stops can be created and fetched per route
Driver-bus assignment	Assign a driver to a specific bus.	FastAPI PUT endpoint	Bus record updated with assigned driver

Table 5: Data models and management API work completed.

4.4 Authentication, Authorization & API Documentation

Security was treated as a first-class concern rather than an afterthought: passwords are never stored in plain text, sessions are stateless via JWT, and role checks are enforced on the server, not just hidden in the UI.

Item	What Was Done	Technology	Current Result
Password hashing	Passwords hashed before storage; never stored as plain text.	bcrypt	Verified: DB never contains raw passwords
JWT authentication	Stateless token issued on login and required on protected routes.	PyJWT / python-jose	Token issued and validated per request
Role-based authorization	Endpoints check the caller's role before executing.	FastAPI dependencies	Non-admins receive 403 on admin routes
Login API	Verifies credentials and issues a JWT.	FastAPI + bcrypt	POST /auth/login returns a working token
Registration API	Creates a new user account.	FastAPI + Pydantic	POST /users/ creates a user

Item	What Was Done	Technology	Current Result
Public Driver/Admin registration blocked	Self-registration restricted to Student role at the API layer.	Server-side role check	Attempting to self-register as Driver/Admin is rejected
Admin-protected APIs	Sensitive endpoints require an authenticated Admin token.	FastAPI dependencies	Verified via GET /users/admin-test
Swagger/OpenAPI	Auto-generated interactive API docs.	FastAPI built-in	Available and used for manual testing

Table 6: Authentication, authorization, and API documentation work completed.

> *Authentication and authorization were documented as implemented in earlier weeks. As good practice, a short regression check of login and role-based access is recommended before each new submission, rather than re-asserting the same status without a fresh check (see Chapter 19.4).*

4.5 Frontend

The frontend work completed so far focuses on the public-facing pages and the student and driver experience. Faculty and admin interfaces are planned next (see Chapter 17).

Item	What Was Done	Technology	Current Result
Landing page	Public entry page introducing the system.	HTML/CSS/JS	Loads and links to login
Login page	Form for email + password, calls the login API.	HTML/CSS/JS + fetch	Successful login redirects to dashboard
Student dashboard	Main authenticated view for students.	HTML/CSS/JS	Loads after login with a token
Single-page navigation	Dashboard sections switch without full page reloads.	Vanilla JS DOM manipulation	Smoother in-dashboard navigation
Live tracking map UI	Map container and layout built into the dashboard.	HTML/CSS	Map renders; no live bus data yet
Leaflet + OpenStreetMap	Interactive map library integrated and initialised.	Leaflet.js + OSM tiles	Map pans/zooms with OSM tiles
Driver dashboard screens	Current-trip and trip-history views built and demonstrated.	HTML/CSS/JS	Screens render sample trip data; see Chapter 19.5

Table 7: Frontend work completed.

> *The live tracking map renders correctly and is ready to receive data, but it is not yet connected to any real bus position — see Chapter 16 for status.*

5. Backend Development

The backend follows a layered folder structure so that request handling, validation, business logic, and data access remain separate and easy to maintain as the codebase grows:

```
backend/ └── app/      ├── models/      (SQLAlchemy ORM table definitions)
├── schemas/     (Pydantic request/response validation)      ├── routers/
(FastAPI route/endpoint definitions)      ├── services/    (business logic,
called by routers)      ├── database/    (DB session/engine setup)      └──
main.py      (FastAPI app entry point)
```

5.1 Purpose of Each Folder

Folder	Purpose
models/	Defines database tables as SQLAlchemy ORM classes (Users, Students, Drivers, Buses, Routes, Stops, etc.).
schemas/	Defines Pydantic models used to validate incoming request bodies and shape outgoing JSON responses.
routers/	Declares the actual API endpoints (paths, HTTP methods) and delegates to services. student.py holds the student-facing routes discussed in Chapter 19.
services/	Contains the business logic — e.g. checking role, hashing a password, assigning a driver — kept separate from the HTTP layer.
database/	Sets up the SQLAlchemy engine, session factory, and dependency used to obtain a DB session per request (connection.py).
main.py	Creates the FastAPI app instance, registers routers, and starts the application.

Table 8: Purpose of each backend folder.

5.2 How a Request Moves Through the Backend

Every implemented API call follows the same layered path, which keeps the codebase predictable and testable:

1. A request reaches FastAPI's ASGI server (Uvicorn).
2. FastAPI matches the URL and method to a Router.
3. The Router passes the request body through a Schema for validation.
4. Valid data is handed to a Service, which applies business rules (e.g. role checks, password hashing).
5. The Service reads or writes through the Database layer to PostgreSQL.
6. The result flows back up: Database → Service → Router → JSON Response to the client.

This exact flow is illustrated as a diagram in Chapter 8 (Current Data Flow). A dedicated kafka/ folder now sits alongside backend/ and holds the asynchronous GPS-processing path described in Chapter 19 — that path runs independently of the request/response cycle above.

6. Database

The project uses a single PostgreSQL database named university_bus_tracking. As of Week 5, the table set falls into three groups: core tables created and actively used since early

development; tables that are part of the real-time design and have now been verified to hold real data (trips, bus_locations); and tables that remain designed but not yet populated or read by working application logic.

Table	Purpose	Status	Verified In
users	Base identity table (email, hashed password, role) shared by all four roles.	COMPLETED	Week 1-2
students	Student-specific profile data, linked to a users row.	COMPLETED	Week 1-2
drivers	Driver-specific profile data, linked to a users row.	COMPLETED	Week 1-2
faculty	Faculty-specific profile data, linked to a users row.	COMPLETED	Week 1-2
buses	Bus fleet records (registration, capacity, assigned driver).	COMPLETED	Week 1-2
routes	Named routes that buses operate on.	COMPLETED	Week 1-2
stops	Stops belonging to a route, in sequence.	COMPLETED	Week 1-2
trips	A single run of a bus along a route; holds trip status (e.g. ACTIVE) and trip date.	VERIFIED	Week 5
bus_locations	Time-stamped GPS points per bus, written by the Kafka consumer.	VERIFIED	Week 5
subscriptions	Links a student to the route(s) they follow, for personalised tracking/notifications.	PLANNED	—
schedules	Planned/scheduled timing information per route. Currently 0 rows.	PLANNED	—
notifications	Delay/arrival alerts to be pushed to users.	PLANNED	—

Table 9: Database tables, purpose, and implementation status (updated Week 5).

> *"Verified" for trips and bus_locations means real rows were observed during Week 5 testing (an ACTIVE trip and fresh GPS coordinate rows respectively) — it does not mean the surrounding features (trip lifecycle automation, live map, ETA) are complete. schedules, subscriptions, and notifications remain empty/unused tables.*

6.1 Entity-Relationship Diagram

The diagram below shows how the core tables relate to each other. trips and bus_locations are now shown as verified (green) following Week 5 testing; subscriptions, schedules, and notifications remain planned (grey).

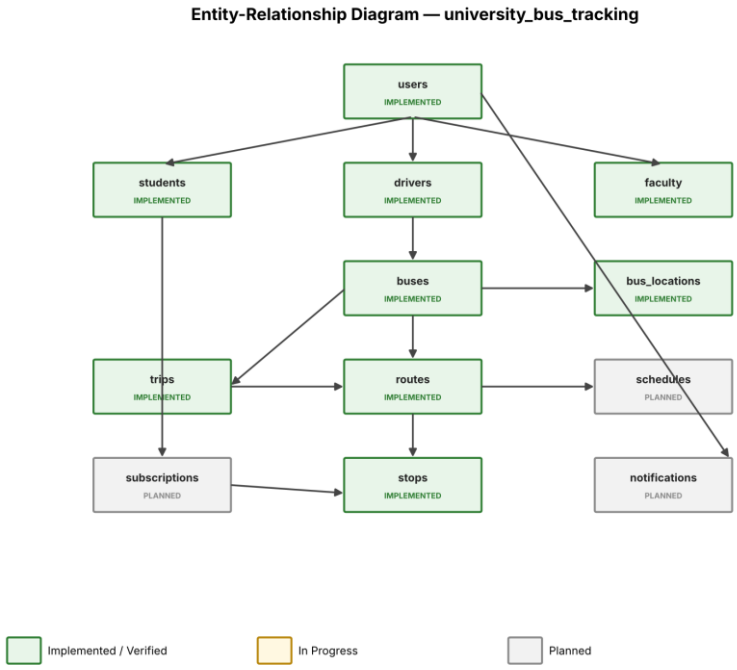


Figure 1: Entity-Relationship diagram for university_bus_tracking, updated Week 5 — implemented/verified vs. planned tables clearly marked.

7. System Architecture

The system architecture is presented in three parts: the implemented request/response path used today, the real-time GPS pipeline (now partially verified — see Chapter 19 for the Week 5 detail), and the planned analytics pipeline. These are kept visually and physically separate so that implemented, verified, and planned work are never confused.

7.1 Implemented Architecture

All currently working features — authentication, and management of students, drivers, faculty, buses, routes, and stops — flow through this path.

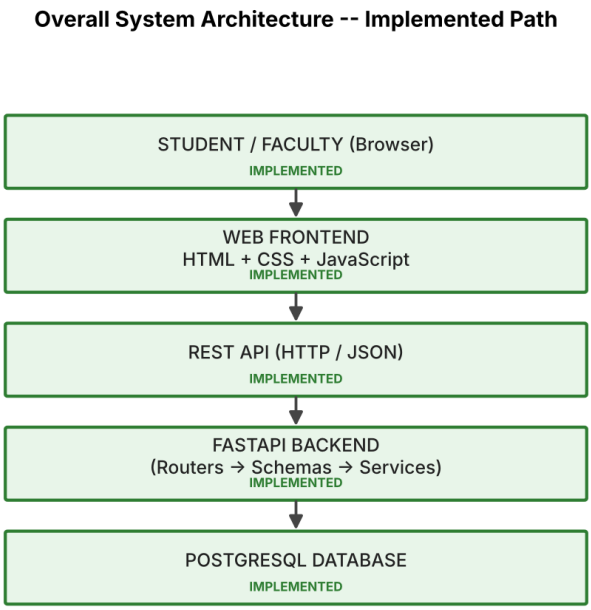


Figure 2: Overall system architecture — the implemented request/response path.

7.2 Real-Time Location Pipeline (Target Architecture, as originally planned)

This pipeline describes how a driver's live GPS position is intended to reach a student's map. The diagram below reflects the original target design from Week 4 and earlier; Chapter 19 documents which specific stages were verified working in Week 5 (Kafka broker, topic, consumer, and database persistence) and which remain pending (driver-side capture, the FastAPI producer endpoint, the live map, and ETA).

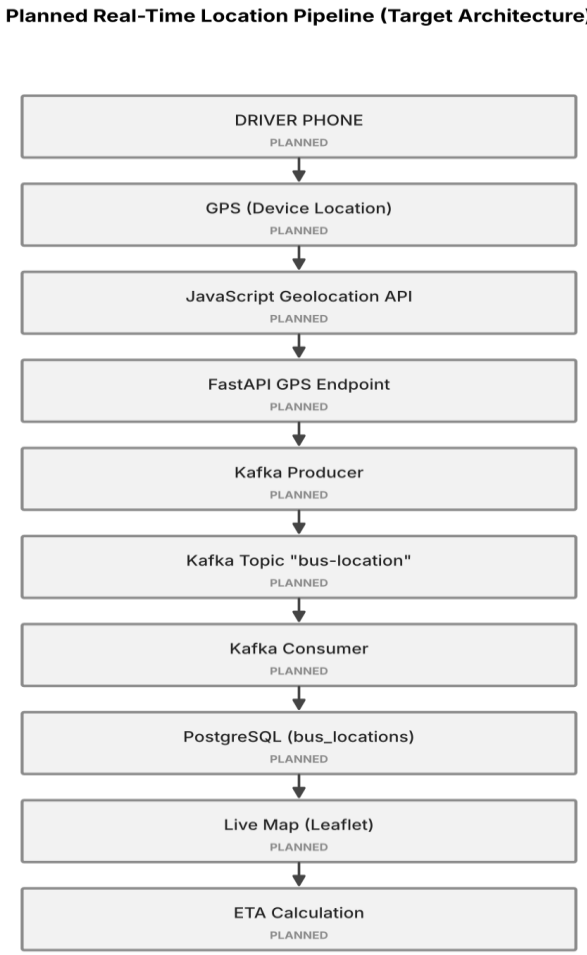


Figure 3: Planned real-time location pipeline — original target architecture.

7.3 Planned Analytics Pipeline

PostgreSQL already holds the operational data Superset would need; the Superset connection and dashboards themselves are planned for a later phase.

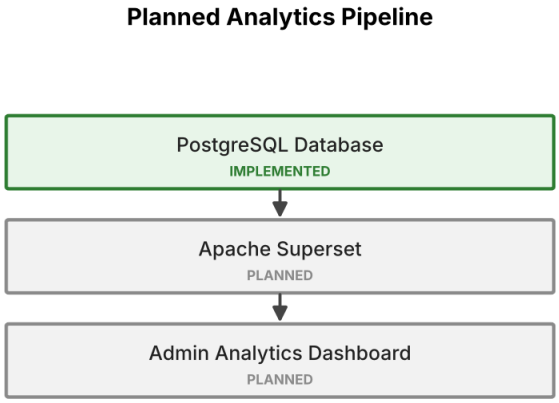


Figure 4: Planned analytics pipeline via Apache Superset.

8. Current Data Flow

The data flow diagram below documents exactly how a request is currently processed end-to-end for any implemented feature (for example, an admin creating a bus, or a student logging in). Every stage shown here is implemented and has been exercised through Swagger and the frontend.

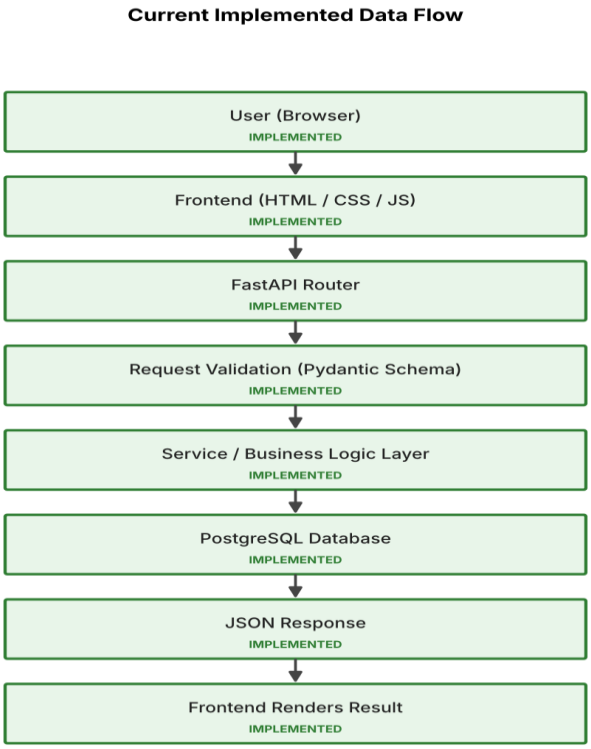


Figure 5: Data flow diagram for the currently implemented system.

In words: the user interacts with the Frontend, which sends an HTTP request to a FastAPI Router. The Router validates the incoming payload against a Schema. Valid requests reach the Service layer, which applies business logic and talks to PostgreSQL. The result is serialised back into JSON and returned to the Frontend, which updates the UI. Note that this is the synchronous request/response path; the Kafka-based GPS path described in Chapter 19 is asynchronous and does not follow this same route.

9. Login Flow

Authentication (proving who the user is) and authorization (deciding what that user may do) are implemented as two distinct steps, which is a deliberate security design choice.

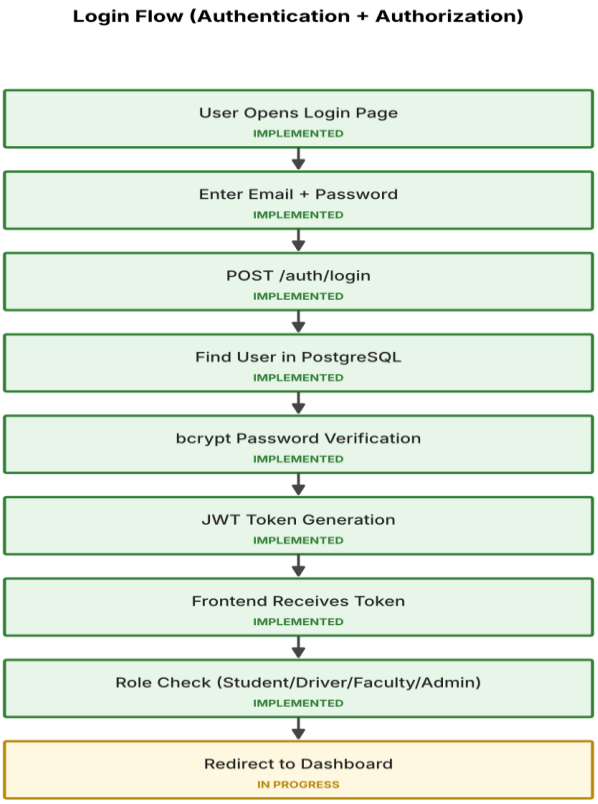


Figure 6: Login flow — authentication and authorization.

9.1 Authentication

The user submits an email and password to POST /auth/login. The backend looks up the user in PostgreSQL, and verifies the submitted password against the bcrypt hash stored for that user. If it matches, a JWT is generated and returned to the frontend. This proves identity only — it does not yet decide what the user can access.

9.2 Authorization

Once the frontend holds a JWT, every subsequent request includes it. Protected endpoints decode the token, read the embedded role claim, and only proceed if that role is permitted for the endpoint (for example, admin-only endpoints reject Student or Driver tokens with a 403 response). This role check happens on the backend, not just by hiding buttons in the UI, which is what makes it a genuine authorization control rather than a cosmetic one.

10. Admin Flow

An admin logs in like any other user, but the resulting JWT carries the Admin role. Every admin API additionally passes through an authorization check before reaching the underlying management logic.

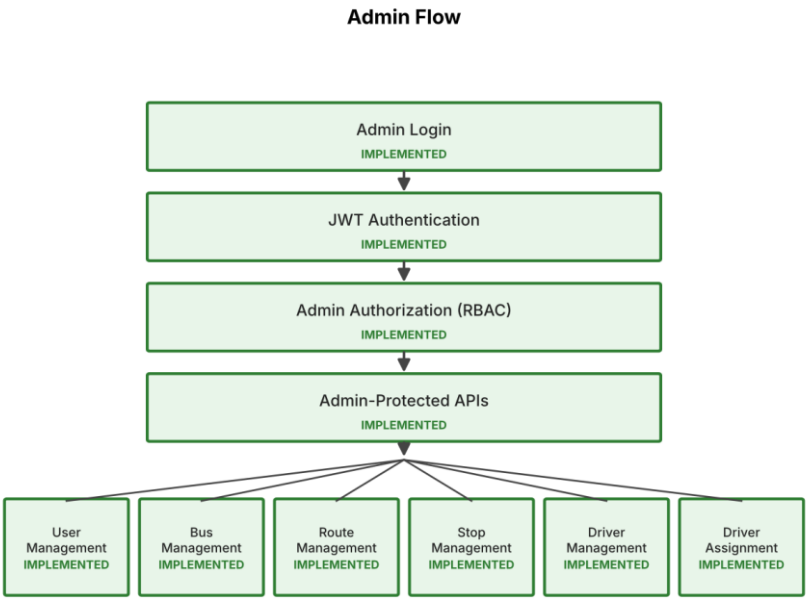


Figure 7: Admin flow: authentication, authorization, and the management APIs it unlocks.

All six management capabilities shown above (User, Bus, Route, Stop, Driver management, and Driver Assignment) are implemented as working backend APIs today. A dedicated Admin dashboard UI to drive them visually is planned (see Chapter 17, Phase 10); until then, these APIs are exercised through Swagger/OpenAPI.

11. Student Flow

The student flow reflects the actual state of the student-facing frontend and APIs. As of Week 5, four student endpoints are verified working (see Chapter 19.4), including bus details for an active trip; timetable and notifications remain pending. This chapter is explicit about which parts are UI-only, which are API-verified, and which are still pending.

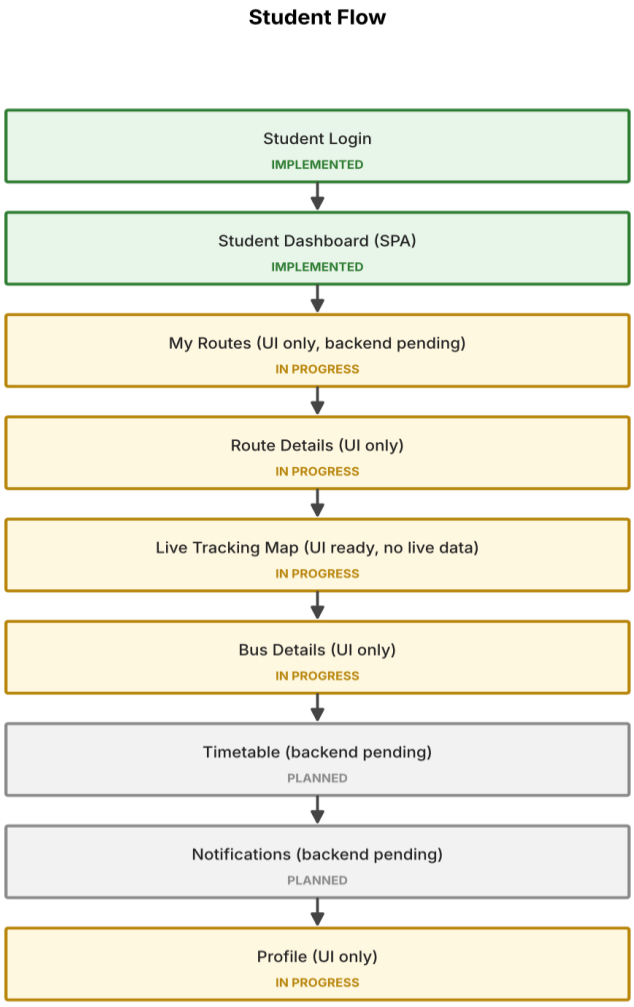


Figure 8: Student flow, with each section marked by its real implementation status.

Section	What Exists Today	Status
Login	Full login form, calls the real login API.	COMPLETED
Dashboard	Single-page dashboard shell with navigation.	COMPLETED
My Routes	UI built; GET /students/me/routes verified 200 OK (Week 5).	VERIFIED
Route Details / Stops	UI built; GET /students/me/routes/5/stops verified 200 OK (Week 5).	VERIFIED
Bus Details	UI built; GET /students/me/routes/5/bus verified 200 OK after Week 5 fix.	VERIFIED
Live Tracking	Map renders with Leaflet + OSM; not connected to live GPS data.	IN PROGRESS
Timetable	Endpoint not found; schedules table has 0 rows.	PLANNED
Notifications	Endpoint returned 404 during testing.	PLANNED
Profile	UI built; GET /students/me verified 200 OK (Week 5).	VERIFIED

Table 10: Student dashboard sections and their verified status as of Week 5.

12. Driver Flow

The driver flow is the planned entry point into the real-time pipeline. As of Week 5, the driver dashboard, current-trip view, and trip-history screens have been demonstrated with sample data (Chapter 19.5), and a driver/bus/trip data record exists and was used to validate the student bus-details fix. The browser-based GPS capture and FastAPI producer stages remain planned.

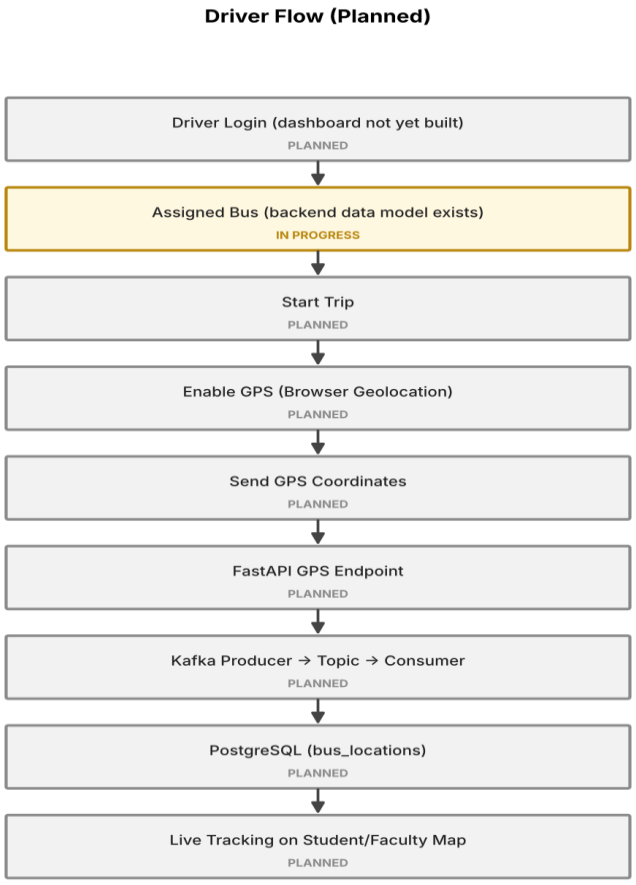


Figure 9: Planned driver flow, from login through to GPS reaching the live map.

> *GPS capture, the FastAPI GPS producer endpoint, and the Kafka-producer side of this flow are PLANNED. The driver dashboard UI and the underlying driver/bus/trip data model are demonstrated or implemented; the Kafka broker, topic, consumer, and database write are VERIFIED (Chapter 19) using a manually published test event rather than a real device GPS signal.*

13. Complete Real-Time System Flow (Target / Planned)

This chapter documents, in detail, how the finished system is intended to work end-to-end. It was written at Week 4 as a planning reference and is kept here unchanged for continuity; Chapter 19 documents exactly which of these stages were verified in Week 5. Readers should treat this chapter as the target design and Chapter 19 as the current ground truth for the GPS/Kafka portion.

13.1 GPS Data Flow

The driver's phone will capture location through the browser's Geolocation API, send it to a dedicated FastAPI endpoint, and that endpoint will publish it onto a Kafka topic rather than writing directly to PostgreSQL — this decouples GPS ingestion speed from database write speed and allows multiple consumers later (e.g. live map, analytics). As of Week 5, the publish-to-consume-to-database half of this diagram is verified; the driver-phone-to-FastAPI-producer half is not yet built (Chapter 19.1–19.2).

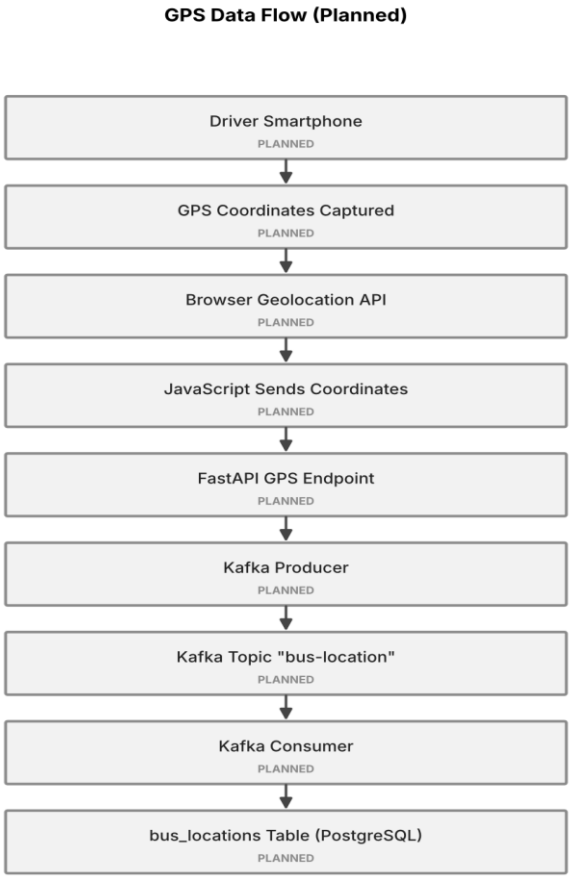


Figure 10: GPS data flow — original target design.

13.2 Kafka Streaming Flow

A Kafka Producer embedded in the GPS endpoint will publish each location update as a message on a topic named "bus-location". A Kafka Consumer subscribes to that topic and persists each update into the bus_locations table. As of Week 5, the topic, the consumer, and the database write are verified (Apache Kafka 4.1.2, KRaft mode, localhost:9092); the producer endpoint embedded in FastAPI is not yet built — test events were published manually during verification. See Chapter 19.2 for full detail and Figure 15 for the updated diagram.

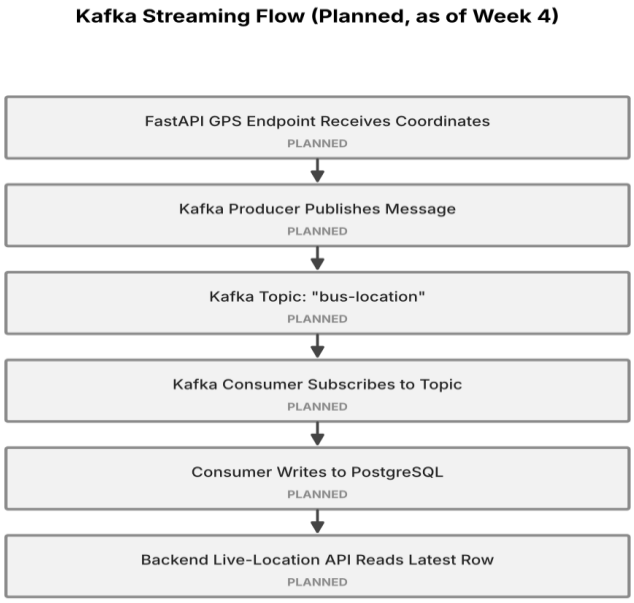


Figure 11: Kafka streaming flow — original target design (see Figure 15 in Chapter 19 for the Week 5 verified version).

13.3 ETA Calculation Flow

Once live positions are available, ETA will be derived (not looked up) from the bus's current GPS point, the next stop on its route, the straight-line/haversine distance between them, and a recent-speed estimate from the last few GPS points. This calculation, and the API that would expose it, are both planned and not yet built. The Week 5 driver dashboard currently displays "Not tracked yet" for next-stop/progress information, consistent with this remaining pending.

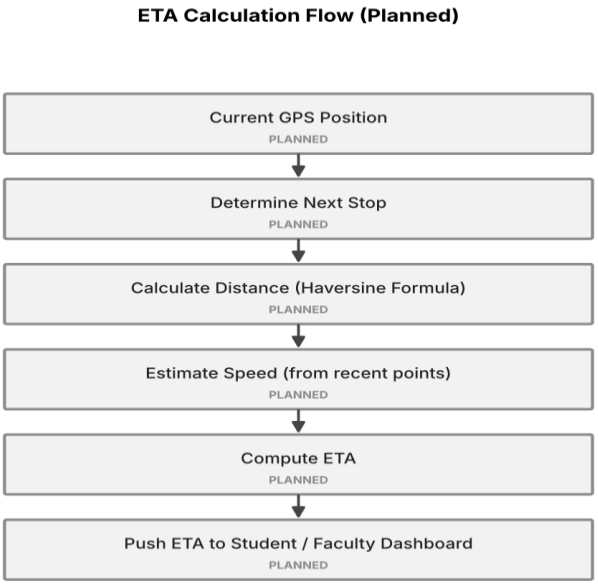


Figure 12: ETA calculation flow — target design, not yet implemented.

13.4 Complete Target Flow (End-to-End)

The diagram below combines the previous three flows into a single end-to-end picture, from the driver's phone to a moving marker on a student's or faculty member's map. It remains the overall target; Chapter 19 narrows in on exactly how far Week 5 progressed toward it.

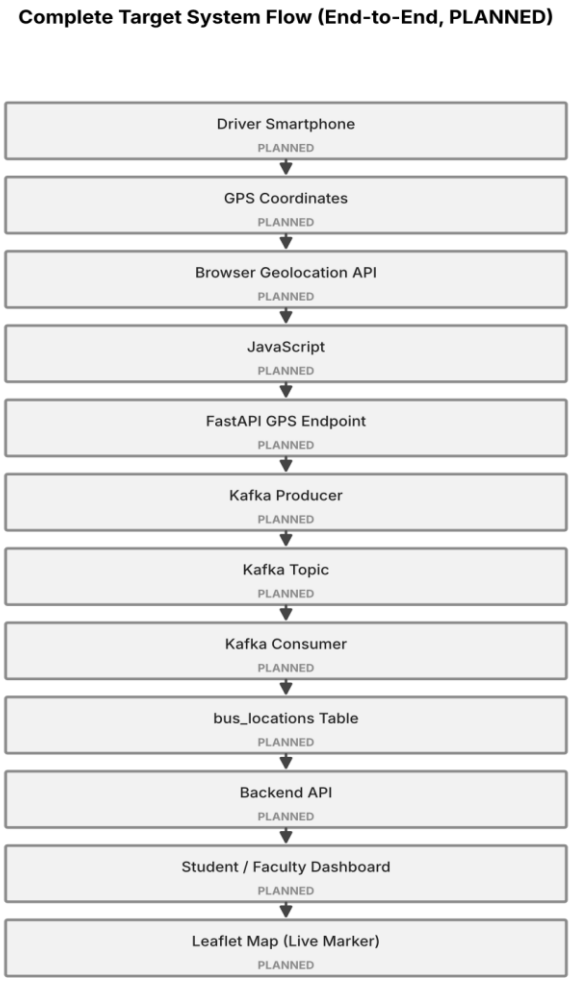


Figure 13: Complete target system flow — end-to-end target architecture.

13.5 Analytics Path

In parallel with live tracking, the same PostgreSQL data (trips, bus_locations, schedules) is intended to feed Apache Superset, giving administrators dashboards on metrics such as on-time trip starts, route usage, and ETA accuracy. This is planned for Phase 11 of the roadmap and has not been started.

14. API Documentation

This chapter documents only the APIs that actually exist and respond, as exercised through Swagger/OpenAPI. Request field names are intentionally omitted where they are not already

fixed in the implementation, to avoid documenting anything speculative. Status reflects the most recent verification; where a result was previously documented but not re-tested this week, it is marked accordingly.

Endpoint	Method	Purpose	Auth Required	Status
GET /	GET	Basic health/welcome check for the API.	No	COMPLETED
GET /db-test	GET	Confirms the backend can reach PostgreSQL.	No	COMPLETED
POST /auth/login	POST	Authenticates a user and issues a JWT.	No	COMPLETED
GET /users/	GET	Lists users.	Yes	COMPLETED
POST /users/	POST	Registers a new user (Student self-registration; Driver/Admin blocked).	No (public, restricted to Student role)	COMPLETED
GET /users/profile	GET	Returns the authenticated user's own profile.	Yes	COMPLETED
GET /users/admin-test	GET	Verifies admin-only authorization is enforced.	Yes (Admin)	COMPLETED
POST /users/students	POST	Creates a student record.	Yes (Admin)	COMPLETED
POST /users/drivers	POST	Creates a driver record.	Yes (Admin)	COMPLETED
GET /users/drivers	GET	Lists drivers.	Yes (Admin)	COMPLETED
POST /users/faculty	POST	Creates a faculty record.	Yes (Admin)	COMPLETED
POST /users/buses	POST	Creates a bus record.	Yes (Admin)	COMPLETED
GET /users/buses	GET	Lists buses.	Yes	COMPLETED
POST /users/routes	POST	Creates a route.	Yes (Admin)	COMPLETED
GET /users/routes	GET	Lists routes.	Yes	COMPLETED
POST /users/stops	POST	Creates a stop under a route.	Yes (Admin)	COMPLETED
GET /users/routes/{route_id}/stops	GET	Lists stops belonging to a specific route.	Yes	COMPLETED
PUT /users/buses/{bus_id}/driver	PUT	Assigns a driver to a bus.	Yes (Admin)	COMPLETED
GET /students/me	GET	Returns the authenticated student's own profile.	Yes (Student)	VERIFIED

Endpoint	Method	Purpose	Auth Required	Status
GET /students/me/routes	GET	Lists the routes available to the student.	Yes (Student)	VERIFIED
GET /students/me/routes/{route_id}/stops	GET	Lists stops for a given route.	Yes (Student)	VERIFIED
GET /students/me/routes/{route_id}/bus	GET	Returns bus/trip details for a route's active trip; fixed in Week 5.	Yes (Student)	VERIFIED
GET /students/me/timetable	GET	Intended to return the student's timetable.	Yes (Student)	PLANNED — returned 404 in testing
GET /students/me/notifications	GET	Intended to return student notifications.	Yes (Student)	PLANNED — returned 404 in testing

Table 11: Implemented and verified API endpoints, with purpose, authentication requirement, and status (updated Week 5).

> *No GPS-ingestion/producer, ETA, subscription, timetable, notification, or Superset endpoint is functional yet. The Kafka consumer in Chapter 19 is a standalone script, not an HTTP endpoint.*

15. Testing Completed (Through Week 4)

Testing through Week 4 was manual, performed through Swagger/OpenAPI and the browser, covering every implemented feature at least once. These results are carried forward from earlier revisions of this document and were not all individually re-executed in Week 5; Week 5's own testing evidence is kept separately in Chapter 19.4 so that newly verified and previously documented results are not conflated.

Test ID	Test	Expected Result	Actual Result	Status
T01	FastAPI startup	Server starts without errors	Server started successfully	PASS
T02	Root API (GET /)	Returns 200 with welcome message	200 OK returned	PASS
T03	Database connection (GET /db-test)	Confirms PostgreSQL connectivity	Connection confirmed	PASS
T04	Swagger UI	Interactive docs load at /docs	Docs loaded and usable	PASS
T05	Registration	New user created in database	User record created	PASS
T06	Login	Valid credentials return a JWT	Token returned	PASS
T07	Password verification	Wrong password is rejected	401 returned for bad password	PASS
T08	JWT validation	Protected route accepts a valid token	Access granted with valid token	PASS

Test ID	Test	Expected Result	Actual Result	Status
T09	Admin authorization	Non-admin blocked from admin route	403 returned for non-admin	**PASS**
T10	Student creation	Admin can create a student	Student record created	**PASS**
T11	Driver creation	Admin can create a driver	Driver record created	**PASS**
T12	Faculty creation	Admin can create faculty	Faculty record created	**PASS**
T13	Bus creation	Admin can create a bus	Bus record created	**PASS**
T14	Bus retrieval	GET /users/buses returns bus list	Bus list returned	**PASS**
T15	Route creation	Admin can create a route	Route record created	**PASS**
T16	Route retrieval	GET /users/routes returns route list	Route list returned	**PASS**
T17	Stop creation	Admin can create a stop under a route	Stop record created	**PASS**
T18	Stop retrieval	Stops returned for a given route, in order	Stops returned correctly	**PASS**
T19	Driver assignment	Driver linked to a bus via PUT endpoint	Bus record updated with driver	**PASS**
T20	Frontend login	Login page authenticates and redirects	Redirect to dashboard on success	**PASS**
T21	Student dashboard navigation	SPA sections switch without reload	Navigation works as expected	**PASS**

Table 12: Manual tests performed through Week 4, executed against the implemented backend and frontend.

15.1 Pending Tests (Updated Week 5)

The following were pending at Week 4. Their Week 5 status is noted; full detail is in Chapter 19.

- Live GPS capture from a driver device — still PLANNED
- Kafka producer publishing to the bus-location topic — VERIFIED using a manual test event (Chapter 19.1–19.2); FastAPI producer endpoint still PLANNED
- Kafka consumer writing to bus_locations — VERIFIED (Chapter 19.2)
- End-to-end GPS → Kafka → PostgreSQL pipeline — VERIFIED from Kafka onward; driver-to-Kafka leg still PLANNED
- Dynamic ETA calculation — still PLANNED
- Route subscription — still PLANNED
- Notifications — still PLANNED (student endpoint returned 404 in Week 5 testing)

- Superset analytics — still PLANNED

16. Current Status

The table below is the single most concise summary of project status and is updated every time a feature moves between PLANNED, IN PROGRESS, VERIFIED, and COMPLETED. This revision reflects Week 5 progress.

Feature	Status	Notes
GitHub	COMPLETED	Repository live, commits ongoing
Project Structure	COMPLETED	backend/frontend/database/kafka/superset folders in place
PostgreSQL	COMPLETED	Installed, running, connected
Database Schema	COMPLETED	Core tables created; trips and bus_locations now hold verified data
FastAPI	COMPLETED	App running via Uvicorn
Database Connection	COMPLETED	SQLAlchemy wired to PostgreSQL
Authentication	COMPLETED	bcrypt + JWT working (recommend regression check each week)
Authorization	COMPLETED	Role-based checks enforced server-side
Registration	COMPLETED	Public Student registration; Driver/Admin blocked
User Management	COMPLETED	Core user CRUD via API
Student Management	COMPLETED	API implemented
Driver Management	COMPLETED	API implemented
Faculty Management	COMPLETED	API implemented
Bus Management	COMPLETED	API implemented
Route Management	COMPLETED	API implemented
Stop Management	COMPLETED	API implemented
Driver Assignment	COMPLETED	API implemented
Landing Page	COMPLETED	Built and working
Login Page	COMPLETED	Built and working, calls real API
Student Dashboard	COMPLETED	SPA shell with navigation built
Student Bus-Details API	VERIFIED	Fixed and verified Week 5 — returns 200 OK for active trip
Map UI	IN PROGRESS	Leaflet + OSM rendering; no live data
Driver Dashboard Screens	IN PROGRESS	Current-trip and trip-history views demonstrated
Kafka Broker (KRaft)	VERIFIED	Running on localhost:9092, Week 5
Kafka Topic (bus-location)	VERIFIED	Created and inspected, Week 5
GPS Consumer	VERIFIED	Receives events and persists to PostgreSQL, Week 5
Driver GPS Capture	PLANNED	Browser Geolocation capture not started

Feature	Status	Notes
FastAPI GPS Producer Endpoint	PLANNED	Not started; Week 5 events were published manually
Real-Time Pipeline (end-to-end)	IN PROGRESS	Kafka-to-database half verified; driver-to-Kafka half pending
Dynamic ETA	PLANNED	Not started
Timetable	PLANNED	schedules table empty; endpoint returns 404
Notifications	PLANNED	Student endpoint returns 404
Subscriptions	PLANNED	Table designed only
Faculty Dashboard	PLANNED	Not started
Admin Dashboard UI	PLANNED	APIs exist; no UI yet
Admin CRUD for Full Fleet	PLANNED	9-bus fleet not yet fully configured
Superset	PLANNED	Folder reserved; not connected
Deployment	PLANNED	Currently local development only

Table 13: Feature-by-feature project status as of Week 5.

17. Development Roadmap

The roadmap is organised into twelve phases. Phases 1–4 are complete. Phase 5 (Driver GPS) and Phase 6 (Kafka) are now IN PROGRESS following Week 5 work — Kafka's consumer side is verified, while the driver-side producer is not yet built. Phases 7–12 remain PLANNED.

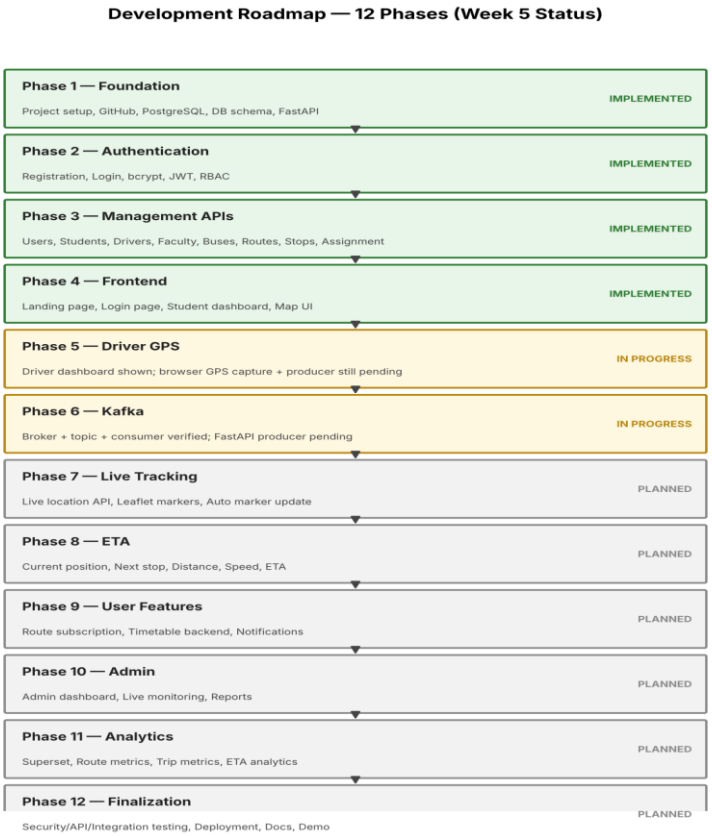


Figure 14: Twelve-phase development roadmap, updated with Week 5 progress.

Phase	Scope	Status
Phase 1 — Foundation	Project setup, GitHub, PostgreSQL, database schema, FastAPI.	COMPLETED
Phase 2 — Authentication	Registration, Login, bcrypt, JWT, role-based access control.	COMPLETED
Phase 3 — Management APIs	Users, Students, Drivers, Faculty, Buses, Routes, Stops, Driver assignment.	COMPLETED
Phase 4 — Frontend	Landing page, Login page, Student dashboard, Map UI.	COMPLETED
Phase 5 — Driver GPS	Driver dashboard shown (Week 5); browser GPS capture and producer still pending.	IN PROGRESS
Phase 6 — Kafka	Broker, topic, consumer, and PostgreSQL write verified (Week 5); FastAPI producer pending.	IN PROGRESS
Phase 7 — Live Tracking	Live location API, Leaflet markers, automatic marker update.	PLANNED
Phase 8 — ETA	Current position, next stop, distance, speed, ETA computation.	PLANNED
Phase 9 — User Features	Route subscription, timetable backend, notifications.	PLANNED
Phase 10 — Admin	Admin dashboard UI, live monitoring, reports, full 9-bus fleet configuration.	PLANNED
Phase 11 — Analytics	Superset integration, route/trip metrics, ETA analytics.	PLANNED
Phase 12 — Finalization	Security/API/integration testing, deployment, final documentation, PPT, demo.	PLANNED

Table 14: Development roadmap phases and current status (updated Week 5).

18. What We Should Do Next

The following ordered list is the immediate, actionable task queue for the team following Week 5. It supersedes the Week-4 version of this list and will be trimmed and updated again as tasks are completed.

1. Build the FastAPI GPS producer endpoint that publishes a real event onto the bus-location Kafka topic.
2. Implement driver-side GPS capture using the browser Geolocation API.
3. Test GPS capture from an Android phone end-to-end into Kafka.
4. Handle the null-speed case in the consumer/ETA logic explicitly (data-quality rule, not a silent default).
5. Define a trip lifecycle policy so stale ACTIVE trips are closed automatically, removing the current query trade-off (Chapter 19.3).
6. Connect the latest row in bus_locations to the Leaflet map so a bus marker updates live.
7. Implement ETA calculation (distance to next stop, recent speed, arrival estimate).

8. Populate the schedules table with official timetable data and implement the /students/me/timetable endpoint.
9. Implement the /students/me/notifications endpoint and the underlying notification-generation logic.
10. Build the Admin CRUD dashboard UI and use it to configure the complete 9-bus fleet (6 route buses, 3 shuttles).
11. Build the faculty dashboard.
12. Integrate Apache Superset.
13. Re-run and screenshot the full regression suite (Chapters 15 and 19.4) before the next submission.
14. Carry out final security, API, and integration testing.
15. Deploy the system.

> *This document will continue to be revised alongside development. Each future weekly update should move newly verified items into the relevant earlier chapter, add a new dated chapter for that week's work (following the pattern of Chapter 19), and refresh Chapters 16 and 17.*

19. Week 5 — Kafka Integration, GPS Data Persistence, and Student API Improvements

This chapter records the development work carried out after Week 4. It follows the same COMPLETED / IN PROGRESS / VERIFIED / PLANNED / NOT VERIFIED labelling used throughout this document (see the Document Purpose page). The headline result of Week 5 is that the Kafka-to-database half of the real-time location pipeline — broker, topic, consumer, and PostgreSQL persistence — has been verified working together for the first time. The driver-to-Kafka half (GPS capture and the FastAPI producer endpoint) was not built this week and remains planned, as does the student-facing live map and ETA. A student API defect was also found, diagnosed, and fixed.

19.1 Kafka Environment Setup

Apache Kafka 4.1.2 was installed and run locally inside a Ubuntu WSL environment, using Kafka's KRaft mode rather than the older ZooKeeper-based mode.

What KRaft Is and Why It Was Used

KRaft (Kafka Raft) is Kafka's built-in metadata-management mode, in which the Kafka brokers themselves use the Raft consensus protocol to agree on cluster metadata (topics, partitions, configuration), instead of relying on a separate ZooKeeper ensemble. For a two-person student project running Kafka on a single local machine, KRaft is the simpler and more current choice:

there is one fewer service to install, configure, and keep running, and it reflects the direction Kafka itself has moved in.

Broker, Topic, and Partitions

A Kafka broker is the server process that stores and serves messages. A topic is a named stream of messages — here, bus-location — that producers publish to and consumers read from. A topic is split into one or more partitions, which are the unit of parallelism and ordering: messages within a single partition are strictly ordered, but there is no ordering guarantee across different partitions of the same topic. The replication factor (RF) controls how many broker copies each partition has, for fault tolerance if a broker fails.

Setting	Value Used and Why
Broker address	localhost:9092 — a single local broker, sufficient for development and testing on one machine.
Storage initialisation	Local Kafka storage metadata was found to be missing and was reinitialised before the broker would start.
Topic name	bus-location — one topic carries all GPS location events for this stage of development.
Partitions	1 — sufficient for a single-bus development test; ordering of events is guaranteed since there is only one partition.
Replication factor	1 — no redundant copy of the data exists on another broker.

Table 15: Week 5 Kafka configuration and rationale.

> *This single-broker, single-partition, RF=1 configuration is appropriate for local development only. It is not a highly-available production configuration: with RF=1, each partition has no redundant replica on another broker, so if the single broker goes down, the bus-location topic becomes unavailable to producers and consumers for as long as that broker is down — and if the broker's underlying storage is permanently lost (disk failure, deleted data directory) rather than just temporarily offline, any messages that had not yet been consumed are lost with it, since no other broker held a copy. With one partition, message throughput also cannot be parallelised across multiple consumer instances. A production deployment would need multiple brokers, a replication factor of at least 3, and a partition count chosen for expected throughput (e.g. one partition per bus or per route, depending on required ordering). This is noted here as a known limitation, not as something to fix within the current academic timeline unless the roadmap calls for it.*
>
> *Data preservation note: reinitialising local Kafka storage discarded whatever test messages existed in that local Kafka log — Kafka's storage, not PostgreSQL. The PostgreSQL database was not reset, dropped, or truncated at any point, and all previously recorded bus locations, trips, and other rows remain intact. No DROP, TRUNCATE, or schema-reset command was run against PostgreSQL during this work.*

19.2 GPS Consumer and PostgreSQL Integration

The data flow verified this week is: a GPS event is published to the Kafka bus-location topic, the GPS consumer script (kafka/gps_consumer.py) subscribes to that topic and receives the event, and the consumer then writes a corresponding row into the PostgreSQL bus_locations table.

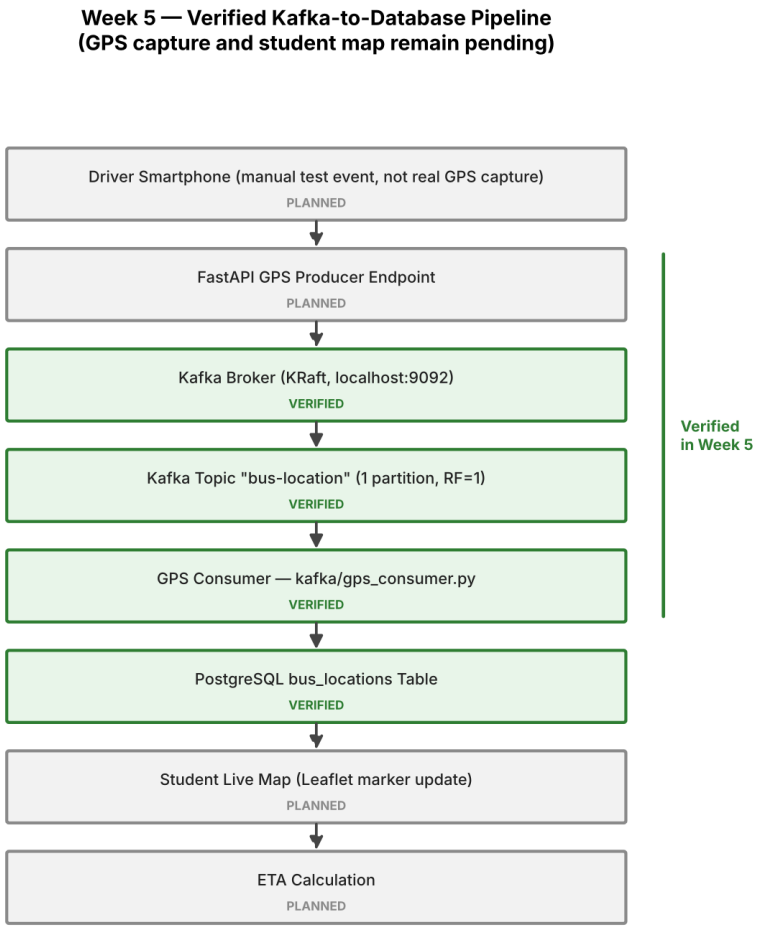


Figure 15: Week 5 verified Kafka-to-database pipeline. The driver-phone, GPS-capture, FastAPI-producer, live-map, and ETA stages remain planned; the Kafka broker, topic, consumer, and database write are verified.

Responsibilities of Each Stage

- GPS event — a message containing fields such as bus_id, trip_id, latitude, longitude, and speed. Observed test events referenced bus BUS-001 and trip_id 2. In this week's testing, events were published as test messages rather than captured from an actual driver device, since the driver-phone-to-FastAPI leg of the pipeline does not exist yet.
- Kafka topic (bus-location) — buffers events between the (not-yet-built) producer and the consumer, decoupling the rate at which events arrive from the rate at which they are written to the database.
- GPS consumer (kafka/gps_consumer.py) — subscribes to the topic, deserialises each event, and is responsible for persistence. During testing it printed each received event to the terminal and logged a confirmation once the corresponding row was saved.
- PostgreSQL bus_locations table — the durable store of GPS history. A read-only query against this table, ordered by recorded_at descending, was used to confirm that new rows were actually being written, not just logged by the consumer.

Data Quality: Null Speed Values

Some received events contained a null value for speed. This is documented here as an open data-quality / optional-field consideration, not as a resolved issue: the consumer currently accepts and stores a null speed rather than rejecting the event or substituting a computed value. Any future ETA calculation that depends on speed (Chapter 13.3) will need an explicit rule for handling a missing speed value — for example, falling back to an average of recent non-null readings — rather than assuming speed is always present.

Why this is a sound design decision: separating event ingestion (Kafka) from database persistence (the consumer, writing to PostgreSQL) means the two can fail, scale, or be restarted independently. If the database were temporarily unavailable, events would still be buffered in the Kafka topic rather than being lost at the source, and the consumer could catch up once the database returns — provided it is restarted before the topic's message retention period expires.

19.3 Student API Debugging and Fix

File affected: backend/app/routers/student.py (the student bus-details endpoint, GET /students/me/routes/{route_id}/bus).

The Problem

The endpoint was returning HTTP 404 for a trip that was, in fact, active. Investigation showed the underlying query was filtering on two conditions at once: it required the trip's stored trip_date to equal the database's CURRENT_DATE, and it restricted which trip statuses it would accept. The test trip was correctly marked ACTIVE, but its stored trip_date was 2026-10-07, while the test was being run on 2026-10-09 — so the date filter excluded an otherwise-valid active trip.

The Fix

The query was changed to prioritise trips whose status is ACTIVE regardless of their stored trip_date, falling back to trips scheduled for the current date if no active trip is found. This reflects the real-world intent of the endpoint better than the original query: a trip actively in progress should be returned to the student even if its date field has not been updated, since the student's practical question is "which bus should I look for right now," not "what was scheduled for today's calendar date." The exact SQL is not reproduced here, as it was not supplied as source code for this revision; it should be pulled directly from the repository (backend/app/routers/student.py) for any reviewer who needs to inspect it line by line.

Observed Result

After the fix, GET /students/me/routes/5/bus returned HTTP 200 with the data for Active Trip #2, where it had previously returned 404.

Design Trade-off and Follow-up Work

> *Accepting any ACTIVE trip regardless of date means that a trip left in ACTIVE status by mistake — for example, if a driver forgot to mark it completed — would continue to be returned indefinitely, potentially showing a student a stale bus. This is a known, accepted trade-off for Week 5, not an oversight: it was the simplest change that fixed the observed bug. A proper trip lifecycle policy (automatically transitioning a trip out of ACTIVE status after a timeout, or when the driver ends the trip) is listed as follow-up work in Chapter 18 and Chapter 17, Phase 5/6.*

19.4 Student API Verification

The following four student endpoints were exercised and their purpose, in the context of the overall system, is explained below.

Endpoint	Purpose / Significance	Result
GET /students/me	Confirms the authenticated student's own identity and profile can be retrieved — the foundation every other student-facing screen depends on.	HTTP 200
GET /students/me/routes	Confirms a student can see which routes are available to them — needed before they can pick one to track.	HTTP 200
GET /students/me/routes/5/stops	Confirms the ordered list of stops for a specific route can be retrieved — needed to show a route's path and to later compute "next stop" for ETA. Route 5's observed test stops were Main Gate, City Mall, and University Gate (development data, not the official route).	HTTP 200
GET /students/me/routes/5/bus	Confirms the active bus/trip for a route can be retrieved — the direct precondition for live tracking. This is the endpoint fixed in Section 19.3.	HTTP 200 (after fix)

Table 16: Student endpoints verified in Week 5 and their role in the overall system.

> *GET /students/me/timetable and GET /students/me/notifications were also tested this week and both returned HTTP 404 — the timetable endpoint was not found in the student router, and the notifications endpoint exists for drivers but not for students. These are documented as PLANNED, not as failed tests of a finished feature, since the feature itself does not exist yet.*

19.5 Driver Dashboard Verification

The driver dashboard, current-trip view, and trip-history screens were demonstrated during testing, showing sample trip records including both active and completed trips. This confirms the screens render and can display trip-shaped data; it does not confirm the full driver workflow

(starting a trip, sharing GPS, ending a trip) is wired end-to-end, because the GPS-sharing half of that workflow has not been built yet (Section 19.1–19.2).

> *The driver dashboard's next-stop/progress indicator currently displays "Not tracked yet." This is reported here honestly as a present limitation, consistent with live tracking and ETA both remaining PLANNED (Chapter 13.3, Chapter 16).*

19.6 Week 5 Testing Evidence

Only results explicitly supported by this week's development notes are marked PASS. Where a supporting screenshot has not yet been captured, that is noted separately in the Evidence column rather than affecting the PASS/FAIL status of the underlying test, since the technical result and the documentation of that result are two different things.

Test ID	Component / Endpoint	Test Objective	Actual Observed Result	Status	Evidence to Capture
W01	Kafka broker (KRaft)	Confirm the broker starts and listens locally	Broker reachable at localhost:9092	**PASS**	Screenshot of broker startup log
W02	bus-location topic	Confirm the topic exists with the intended configuration	Topic inspected: 1 partition, RF=1	**PASS**	Screenshot of topic describe output
W03	GPS consumer	Confirm the consumer receives published events	Consumer printed received event fields	**PASS**	Screenshot of consumer terminal output
W04	PostgreSQL bus_locations	Confirm consumed events are persisted	Fresh rows returned by ORDER BY recorded_at DESC query	**PASS**	Screenshot of SQL query result
W05	GET /students/me	Confirm student profile retrieval	HTTP 200 returned	**PASS**	Screenshot of Swagger response
W06	GET /students/me/routes	Confirm student route listing	HTTP 200 returned	**PASS**	Screenshot of Swagger response
W07	GET /students/me/routes/5/stops	Confirm stop listing for a route	HTTP 200 returned	**PASS**	Screenshot of Swagger response
W08	GET /students/me/routes/5/bus (pre-fix)	Reproduce the original defect	HTTP 404 for an active trip	**PASS (defect confirmed)**	Not required — superseded by W09
W09	GET /students/me/routes/5/bus (post-fix)	Confirm the query fix resolves the defect	HTTP 200 with Active Trip #2 data	**PASS**	Screenshot of Swagger response, 200 OK
W10	GET /students/me/timetable	Check timetable availability	HTTP 404 — endpoint not found	**PLANNED / PENDING**	Not applicable
W11	GET /students/me/notifications	Check student notification availability	HTTP 404	**PLANNED / PENDING**	Not applicable

Test ID	Component / Endpoint	Test Objective	Actual Observed Result	Status	Evidence to Capture
W12	Driver dashboard screens	Confirm current-trip and trip-history views render	Screens displayed sample active/completed trips	PASS (UI only)	Screenshot of driver dashboard
W13	Driver-to-Kafka producer	Confirm a real device can publish a GPS event via FastAPI	Not attempted this week	NOT VERIFIED	N/A — feature not built yet
W14	Student live map marker	Confirm the map updates from live bus_locations data	Not attempted this week	NOT VERIFIED	N/A — feature not built yet

Table 17: Week 5 testing evidence. Expected result for each row is the corresponding implemented behaviour described in the Test Objective column.

19.7 Git and Version Control Workflow

The following is the recommended workflow for this and future weeks' changes. It is a process recommendation, not a record of commits actually made — commit history should be checked directly on GitHub before being cited as evidence (Chapter 11).

1. Create a feature branch for each unit of work (e.g. feature/kafka-gps-consumer, fix/student-bus-details-404) rather than committing directly to main.
2. Make small, focused commits — one logical change per commit — rather than large, mixed commits that bundle unrelated work.
3. Write descriptive commit messages that state what changed and why, for example:
   - fix: return active trip in student bus details endpoint regardless of trip_date
   - feat: add Kafka GPS consumer and persist events to bus_locations
   - docs: add Week 5 progress and testing evidence to project documentation
4. Review the diff (git status / git diff) before every commit to confirm only intended files are staged.
5. Verify the change works locally before merging the feature branch into main.
6. Never commit .env files, JWT secret values, database credentials, the Python virtual environment, or generated local Kafka storage/log data — these should remain in .gitignore.

> *The commit-message examples above are suggested wording for the described changes; they are not a claim that these exact commits exist in the repository. Confirm actual commit history on GitHub before presenting it as evidence.*

19.8 Contribution to the Overall System

Week 5's work closes the database-persistence half of the real-time tracking objective that was PLANNED in every earlier revision of this document (Chapter 2, Chapter 13). Concretely: it

is now possible for a location event, once it reaches Kafka, to end up as a queryable row in PostgreSQL without any manual intervention — which is the piece that Chapter 7's "planned real-time pipeline" and Chapter 13's end-to-end diagram depend on downstream of the driver. It also fixed a real defect in the student-facing API that would otherwise have blocked any future live-tracking UI from ever finding the right trip to display. What remains before the system can show a moving bus to a student is the upstream half (driver GPS capture and the FastAPI producer) and the downstream UI half (reading the latest bus_locations row into the Leaflet map and computing ETA from it) — both are scoped as the next immediate tasks in Chapter 18.

Appendix A — Week 5 Changelog

This changelog lists exactly what was added or changed in this revision of the document, so a mentor reviewing it week-to-week can see the delta without re-reading the whole document.

Area	Change
New chapter	Added Chapter 19, "Week 5 — Kafka Integration, GPS Data Persistence, and Student API Improvements," covering Kafka setup, the GPS consumer/PostgreSQL pipeline, the student bus-details bug fix, Week 5 testing evidence, driver dashboard verification, Git workflow, and overall contribution.
New diagram	Added Figure 15, the Week 5 verified Kafka-to-database pipeline, distinguishing verified stages (broker, topic, consumer, database) from planned stages (driver capture, producer, live map, ETA).
Chapter 2 (Objectives)	Added "Persist GPS events via Kafka" and "Capture GPS from the driver's device" as separate objectives; updated status to IN PROGRESS / PLANNED respectively.
Chapter 3 (Roles)	Added the fleet note: 9 buses (6 route buses, 3 shuttles), with current testing using a single bus/route/trip only.
Chapter 6 (Database)	Updated trips and bus_locations from PLANNED to VERIFIED, reflecting real rows observed in Week 5. Updated the ER diagram (Figure 2) to match.
Chapter 11 (Student Flow)	Updated My Routes, Route Details/Stops, Bus Details, and Profile from IN PROGRESS to VERIFIED, each referencing the specific Week 5 test that confirmed it.
Chapter 12 (Driver Flow)	Added a note on what was and was not verified for the driver side in Week 5.
Chapter 13 (Real-Time Flow)	Added cross-references pointing readers to Chapter 19 for the current, verified state of the Kafka pipeline, while preserving the original target-architecture diagrams unchanged.
Chapter 14 (API Documentation)	Added the four tested student endpoints plus the two pending ones (timetable, notifications) with their Week 5 verification status.
Chapter 15 (Testing)	Retitled to "Through Week 4" and added a note separating it from Week 5's own evidence in Chapter 19.6.
Chapter 16 (Current Status)	Added rows for the Kafka broker, topic, consumer, student bus-details API, and driver dashboard screens; changed Real-Time Pipeline from PLANNED to IN PROGRESS.

Area	Change
Chapter 17 (Roadmap)	Changed Phase 5 (Driver GPS) and Phase 6 (Kafka) from PLANNED to IN PROGRESS; updated the roadmap diagram (Figure 14) to match.
Chapter 18 (Next Steps)	Replaced the task list with the Week 5-informed next steps (FastAPI producer, driver GPS capture, trip lifecycle policy, null-speed handling, live map wiring, ETA, timetable/notifications, Admin CRUD/full fleet configuration).
Table of Contents / List of Figures / List of Tables	Regenerated to include Chapter 19, the new figure, and all new tables, with page numbers verified against the final rendered PDF.

Table 18: Summary of sections added or updated for the Week 5 revision.

Appendix B — Screenshots and Evidence to Capture Before Submission

The technical results in Chapter 19 are documented from development notes, not from attached screenshots. The following evidence should be captured from the actual running environment before this document is submitted to the mentor, so that every VERIFIED claim is backed by a visible artefact.

Evidence ID	What to Capture	Suggested Caption
E01	Kafka broker startup log showing it listening on localhost:9092.	Figure: Kafka broker running in KRaft mode.
E02	Output of a topic-describe command for bus-location (partitions, replication factor).	Figure: bus-location topic configuration.
E03	GPS consumer terminal output showing a received event and a persistence confirmation.	Figure: GPS consumer receiving and saving an event.
E04	Result of the SQL evidence query below, run against bus_locations.	Figure: Latest GPS coordinates stored in PostgreSQL.
E05	Swagger screenshots showing HTTP 200 for each of the four verified student endpoints (Section 19.4).	Figure: Student API endpoints returning 200 OK.
E06	Swagger screenshot of the bus-details endpoint's pre-fix 404 response, if still reproducible, alongside the post-fix 200 response.	Figure: Student Bus Details API before and after the fix.
E07	Driver dashboard current-trip and trip-history screens.	Figure: Driver dashboard showing active and completed trips.
E08	GitHub commit history / repository view for this week's changes.	Figure: Version control history for Week 5.

Table 19: Screenshot and evidence checklist for Week 5 submission.

Suggested read-only SQL evidence query for E04 (run against PostgreSQL; does not modify data):

```
SELECT location_id, trip_id, bus_id, latitude, longitude, speed,
recorded_at FROM bus_locations ORDER BY recorded_at DESC LIMIT 10;
```

> *Only attach screenshots actually captured from the development environment. If a piece of evidence is missing at submission time, list it here as outstanding rather than fabricating a result — this preserves the accuracy this document has maintained throughout.*

Appendix C — Security and Data-Handling Reminders

- Do not include database passwords, JWT secret values, access tokens, or .env file contents in this document or in any screenshot.
- Do not reset, drop, or truncate PostgreSQL data in order to produce cleaner-looking evidence.
- Clearly mark development/test records as such wherever they appear in evidence, so they are not mistaken for official fleet or route data.
- Do not commit .env files, JWT secrets, database credentials, virtual environments, or generated local Kafka storage data to version control.

Appendix D - Project Implementation Screenshot Evidence

- **Instructions**: This appendix is appended to the existing 46-page report. Each bordered area is a placeholder for an actual screenshot. Replace each placeholder with the corresponding real screenshot before final submission.
- **Security**: Never include passwords, .env values, JWT/access tokens, or private credentials in screenshots.

D.1 – Infrastructure and GPS Data Pipeline

E01 – Kafka Broker and Cluster Verification

What to capture: Capture the terminal showing Kafka running and the cluster ID command/output.



Figure E01: Kafka broker and cluster verification.

Evidence note: Supports that the broker is running and reachable.

E02 – Kafka Topic Verification

What to capture: Capture the bus-location topic description, partition and replication details.

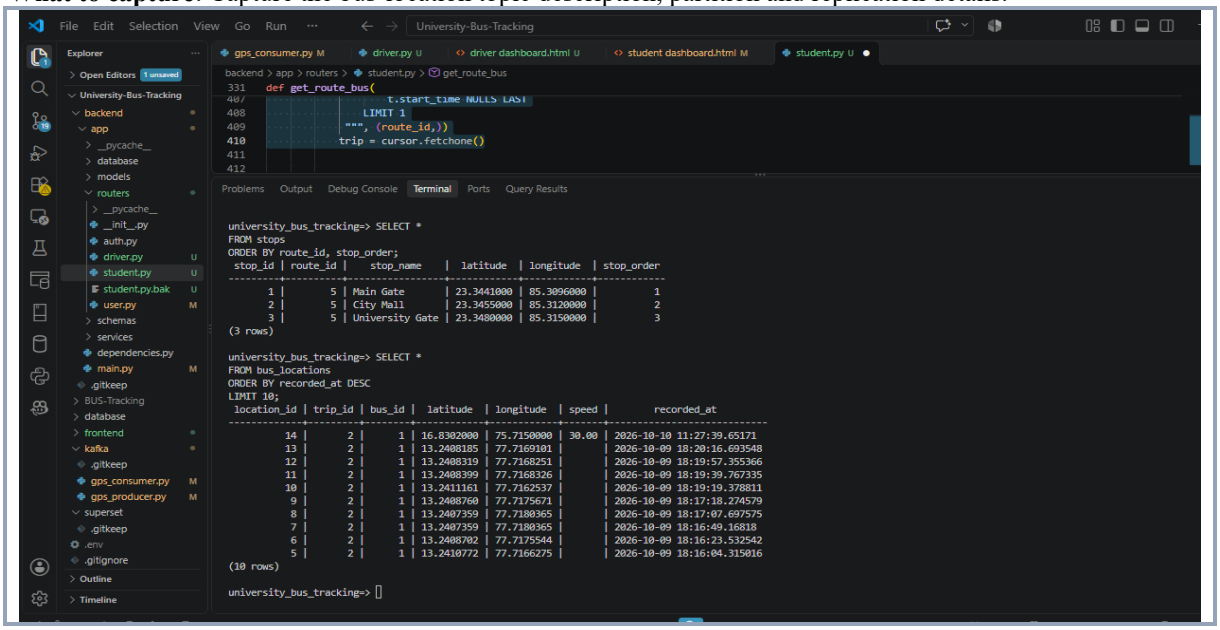


Figure E02: Kafka bus-location topic configuration.

Evidence note: Supports topic creation/configuration.

E03 – Kafka GPS Consumer Processing

What to capture: Capture “GPS Consumer started”, “Kafka GPS received”, and “GPS saved”.



Figure E03: Kafka GPS consumer receives and saves a test GPS event.

Evidence note: Test event processing was observed; this alone does not prove the complete student live map pipeline.

E04 – PostgreSQL GPS Records

What to capture: Capture recent bus_locations rows with latitude, longitude, speed and recorded_at.

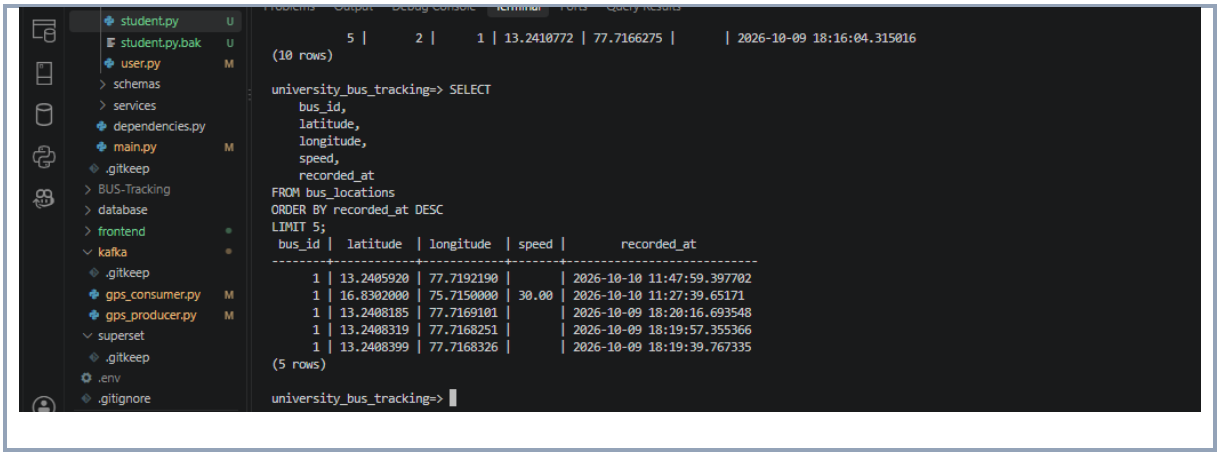


Figure E04: Recent GPS test records persisted in PostgreSQL.

Evidence note: Coordinates may be test data; do not describe them as verified real-world bus coordinates.

D.2 – Student API Verification

E05 – Student Profile API

What to capture: Swagger GET /students/me showing HTTP 200; keep authorization token hidden.

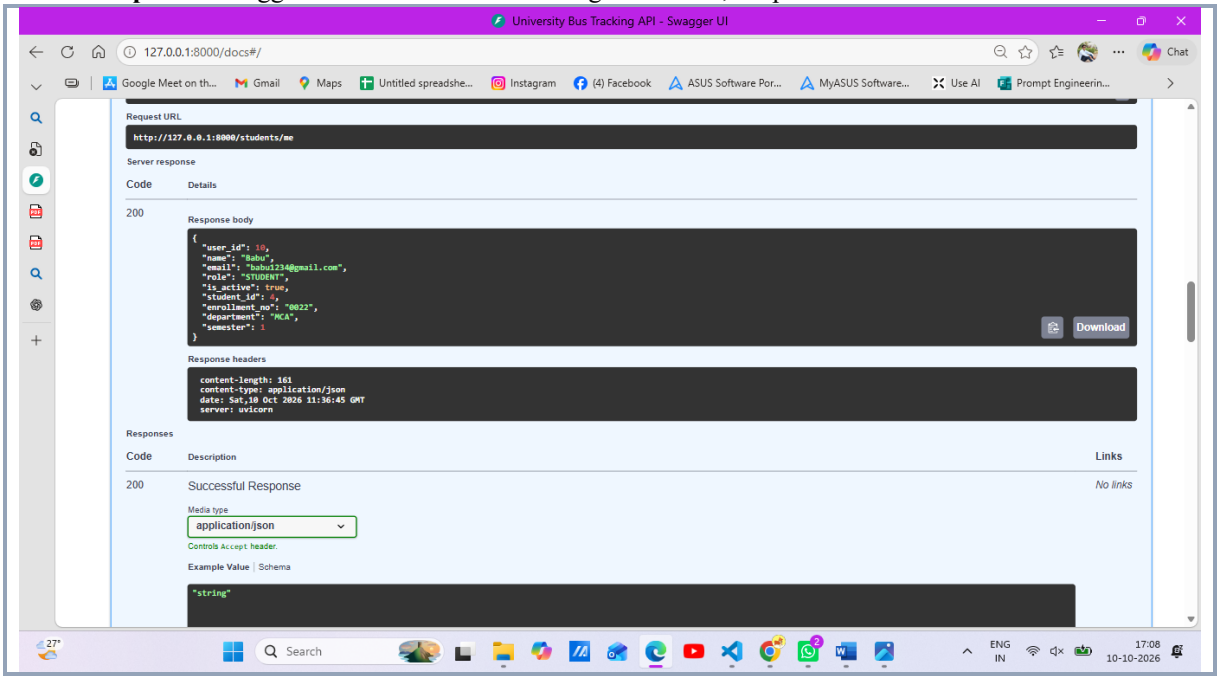


Figure E05: Student profile API returns HTTP 200 OK.

Evidence note: Student profile endpoint verified.

E06 – Student Routes API

What to capture: Swagger GET /students/me/routes showing route_id 5 and route R002.

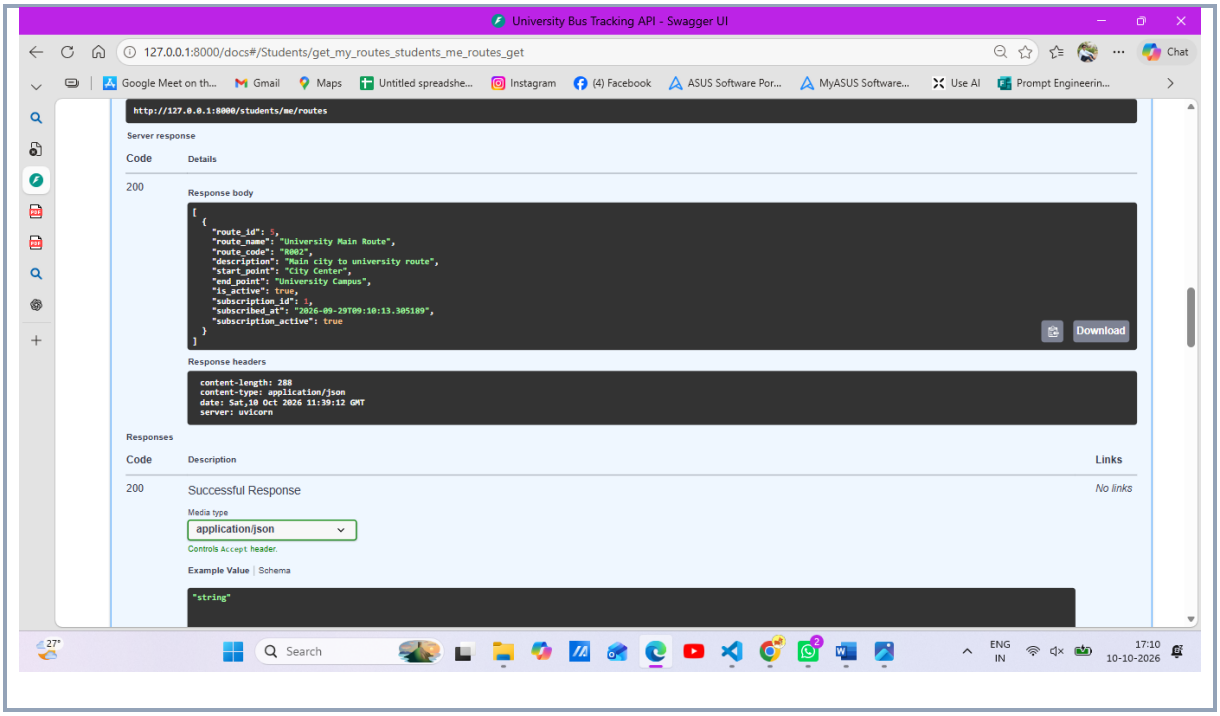


Figure E06: Student routes API returns the subscribed route.

Evidence note: Route list retrieval verified.

E07 – Student Route Stops API

What to capture: Swagger GET /students/me/routes/5/stops showing the ordered stops.

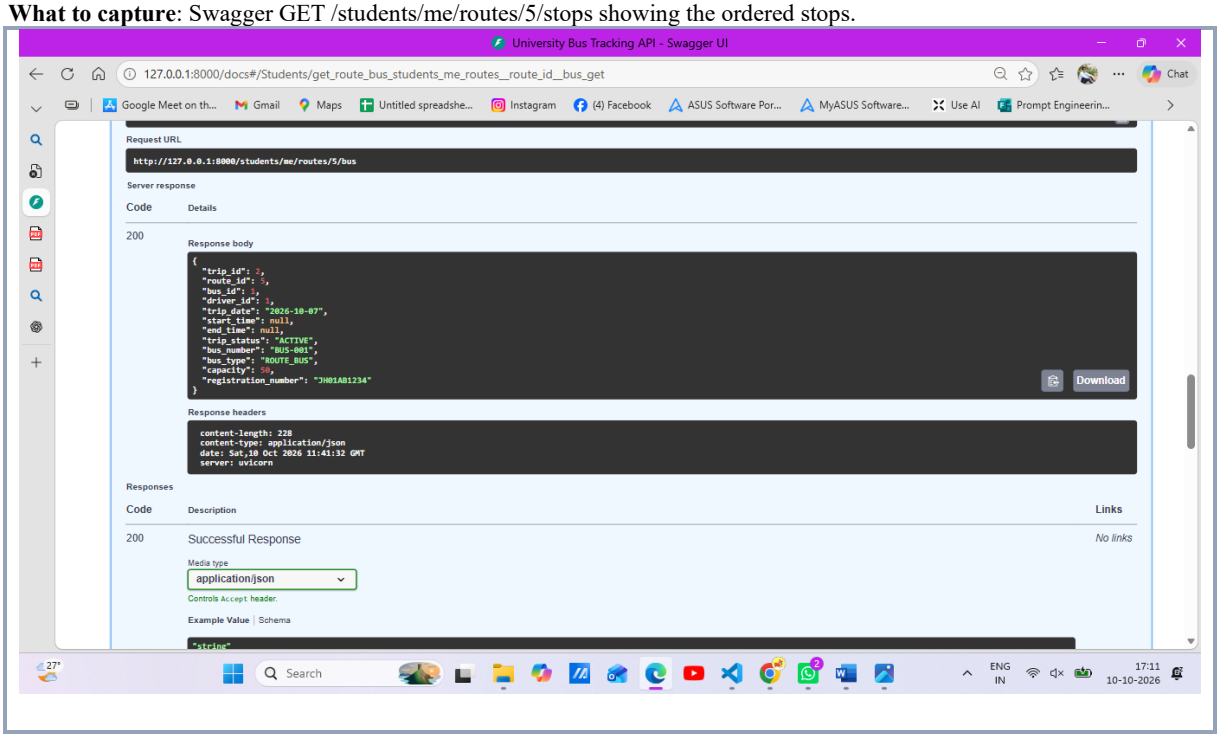


Figure E07: Student route stops API returns ordered stops.

Evidence note: Route-stop endpoint verified.

E08 – Student Bus Details API

What to capture: Swagger GET /students/me/routes/5/bus showing BUS-001 and trip details.

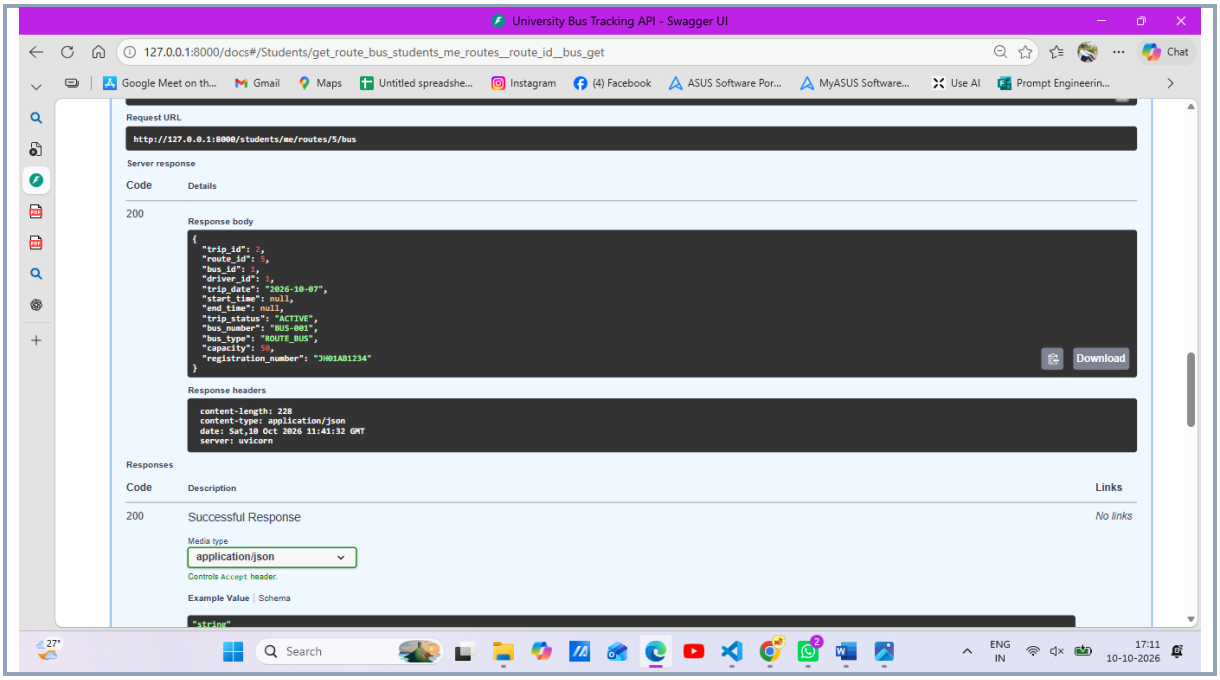


Figure E08: Student bus details API returns BUS-001 trip information.

Evidence note: Bus details endpoint verified.

D.3 – Driver Interface and GPS Sharing

E09 – Driver Dashboard

What to capture: Capture assigned bus, route, trip status, GPS status and stop-progress fields.

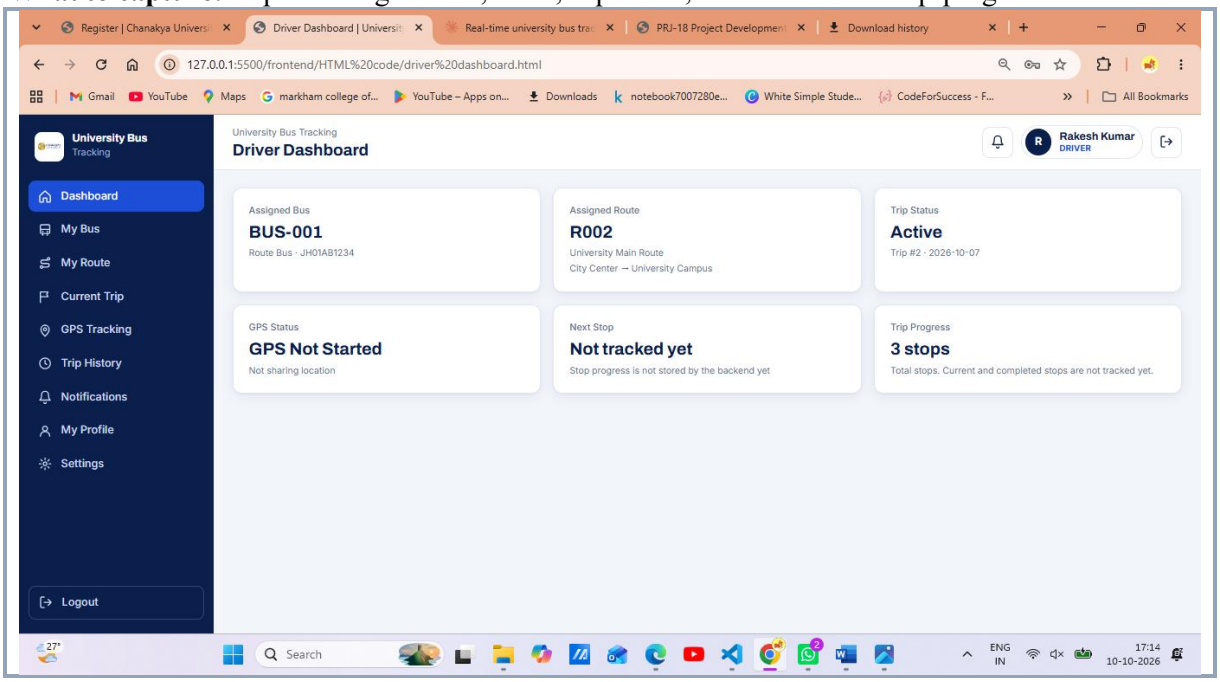


Figure E09: Driver dashboard displays assigned bus, route, and trip status.

Evidence note: Captured view showed GPS not started / stop progress not tracked.

E10 – Driver GPS Tracking

What to capture: Capture GPS status, latitude, longitude, last-updated time and server message.

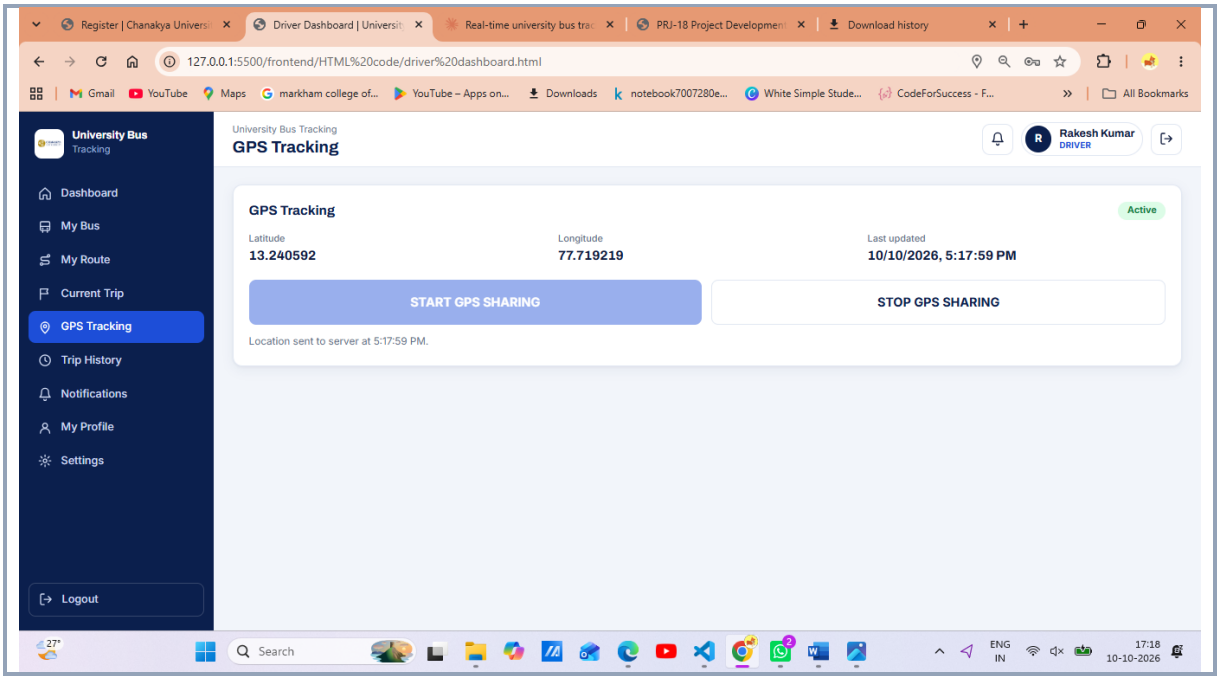


Figure E10: Driver GPS tracking interface reports a location update.

Evidence note: This screen alone does not prove continuous end-to-end live tracking.

D.4 – Student Dashboard and Live Tracking

E11 – Student Dashboard

What to capture: Capture route count, active bus, next-arrival card and map area.

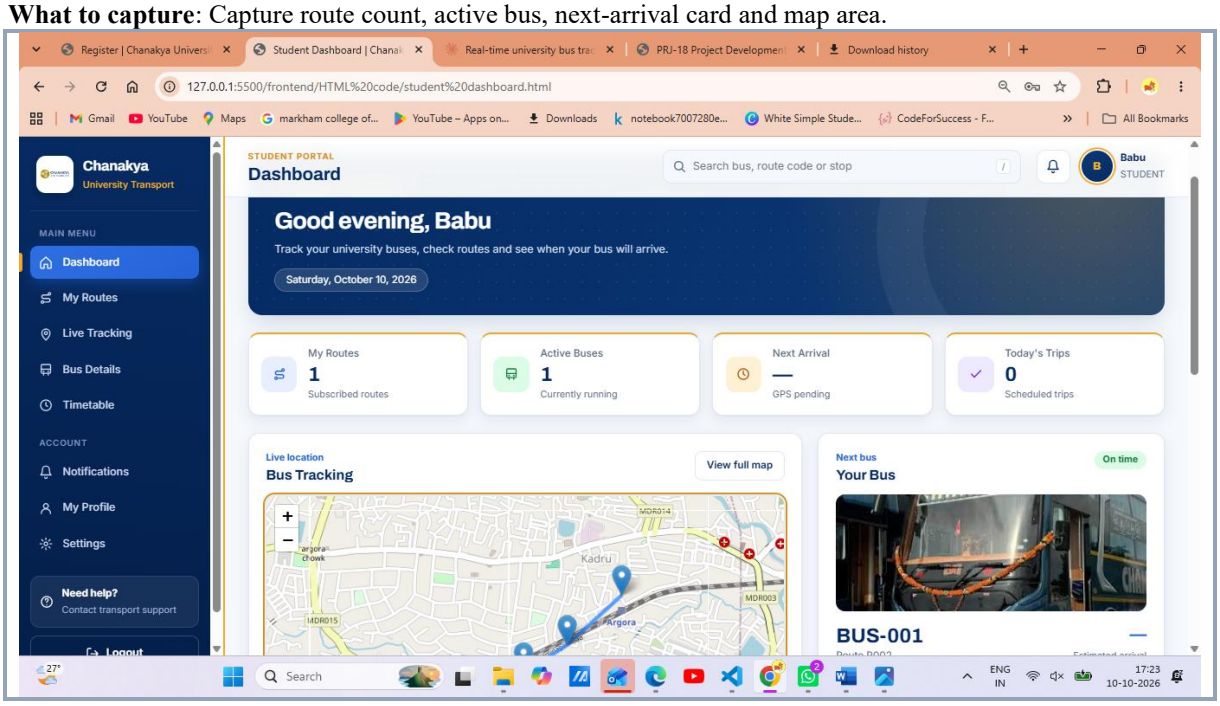


Figure E11: Student dashboard summary and map interface.

Evidence note: A visible map is not proof that its marker uses live GPS.

E12 – Student Live Tracking

What to capture: Capture map and bus card showing current stop, next stop and ETA fields.

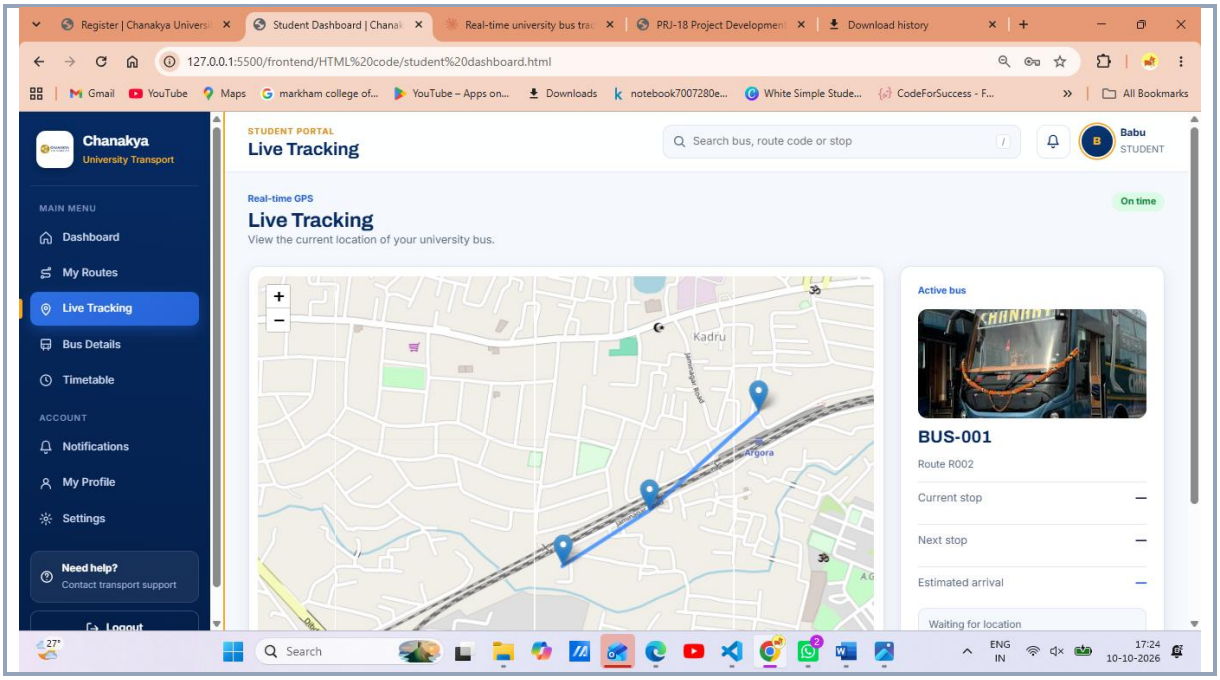


Figure E12: Student live tracking interface and current GPS integration status.

Evidence note: Current/next stop and ETA were blank; frontend GPS polling is disabled and integration remains in progress.

Implementation Status Notes

Verified during the current test session: Kafka consumer displayed a test GPS event as saved; recent GPS rows were visible in PostgreSQL; student profile, routes, route stops and bus details endpoints returned HTTP 200.
Still in progress: student map consuming live GPS updates. The frontend code has GPS polling disabled; current stop, next stop and ETA are not verified as live values. Timetable and notifications requests returned HTTP 404.
Do not describe a complete real-time end-to-end GPS pipeline or dynamic ETA as finished until the student map is confirmed to update from fresh GPS data.